

many requests for small data records. SPTF is more appropriate for systems that generate requests for both large and small data records, because it does not consider the size of a request. 2) Some types of hard disk hide information, such as the location of error-protection data and bad sectors.

12.7 System Considerations

When is disk scheduling useful? When might it degrade performance? These questions must be answered in the context of the overall system. The sections that follow discuss several considerations that might influence a designer's decisions.

Storage as Limiting Resource

When disk storage proves to be a **bottleneck**, some designers recommend adding more disks to the system (see the Operating Systems Thinking feature, Saturation and Bottlenecks). This does not always solve the problem, because the bottleneck could be caused by a large request load on a relatively small number of disks. When this situation is detected, disk scheduling may be used as a means of improving performance and eliminating the bottleneck.

System Load

Disk scheduling might not be useful in a batch-processing system with a relatively low degree of multiprogramming. Scheduling becomes more effective as the randomness of multiprogramming increases, which increases the system load and leads to erratic disk request patterns. For example, file servers in local area networks can receive requests from hundreds of users, which normally results in the kind of ran-



Operating Systems Thinking

Saturation and Bottlenecks

Operating systems often manage hardware and software configurations. When a system is performing poorly, you might be inclined to speak of the problem "in the whole"—the "whole system" is performing poorly. In fact, it's often the case that only one or

a few resources have become saturated, i.e., they have reached their capacity and cannot service requests any faster. Such resources are called bottlenecks. Locating bottlenecks and dealing with them by adding resources in only one or a few areas of the system,

could result in overall system improvement at a relatively modest cost. Systems should be built in ways that make it easy to locate bottlenecks. Chapter 14 presents a thorough treatment of performance issues, including saturation and bottlenecks.

dom request patterns that are best serviced using disk scheduling. Similarly, **online transaction processing (OLTP)** systems, such as Web and database servers, typically receive many disk requests to randomly distributed locations containing small amounts of data (e.g., HTML files and database records). Research has shown that disk scheduling algorithms such as C-LOOK and SATF (with look-ahead) can improve performance in this type of environment.⁴⁴⁻⁴⁵

Much of the analytical work in the literature assumes that request distributions are uniform. Conclusions based on the assumption of uniformity may be invalid on systems whose request distributions are not uniformly distributed over the disk surfaces. **Nonuniform request distributions** are common in certain situations, and their consequences have been investigated.⁴⁶ In one study, Lynch determined that the vast majority of disk references are to the same cylinder as the immediately preceding reference.⁴⁷

One common cause of highly localized nonuniform request distributions is the use of large sequential files on dedicated disks. When an operating system allocates space for the adjacent records of a user's sequential file, it usually places adjacent records on the same track. When a track is full, additional records are placed on adjacent tracks within the same cylinder; when a cylinder is full, additional records are placed on adjacent cylinders. Thus requests for successive records in a sequential file often cause no seeking at all. When needed, seeks are short, because they are usually to immediately adjacent cylinders. Obviously, an FCFS disk scheduling policy would be adequate in this situation. In fact, overhead incurred in more complex scheduling strategies might actually result in degraded performance.

File Organization Techniques

As we discuss in Chapter 13, File and Database Systems, sophisticated file organization techniques can cause a proliferation of requests with large seek times. In some cases, file data retrieval may involve reference to a master index, reference to a cylinder index, and then location of the actual record, a process that could incur several seek delays. Because the master index and cylinder index are normally stored on disk (but separate from the main data area), these seeks can be costly. Such file organization techniques are convenient for the applications designer but can degrade performance.

Self Review

1. Why do many systems exhibit nonuniform request distributions?
2. Why does disk scheduling become more effective as system load increases?

Ans: 1) File data is often stored and accessed sequentially, so a request to a particular cylinder is likely to be followed by a request to the same cylinder, or to an adjacent cylinder. **2)** As system load increases, request patterns tend to become more random, which could lead to

substantial seek activity. Disk scheduling improves performance by reducing the number of wasteful seek operations.

12.8 Caching and Buffering

Many systems maintain a **disk cache buffer**, which is a region of main memory that the operating system reserves for disk data. In one context, the reserved memory acts as a cache, allowing processes quick access to data that would otherwise need to be fetched from disk. The reserved memory also acts as a buffer, allowing the operating system to delay writing modified data until the disk experiences a light load or until the disk head is in a favorable position to improve I/O performance. For example, an operating system may delay writing modified data to disk to allow time for multiple requests to contiguous locations to enqueue, so that they can be serviced with one I/O request.

The disk cache buffer presents several challenges to operating system designers. Because the size of the disk cache must be limited to allow enough memory for active processes, the designer must implement some replacement strategy. The cache-replacement question is similar to the page-replacement question, and designers use many of the same heuristics. Most commonly, designers choose a strategy that replaces the least-recently used item in the disk cache buffer.

A second concern arises because disk caching can lead to inconsistencies. Disk cache buffers are maintained in volatile memory, so if the system fails or loses power while modified data is in the cache buffer, those changes are lost. To safeguard data against such problems, the contents of the disk cache buffer are periodically flushed to the hard disk; this reduces the probability of data loss if the system crashes.

A system that employs **write-back caching** does not write modified data to disk immediately. Instead, the cache is written to disk periodically, enabling the operating system to batch multiple I/Os that are serviced using a single request, which can improve system performance. A system that employs **write-through caching** writes data both to the disk cache buffer and to disk each time cached data is modified. This technique prevents the system from batching requests, but reduces the possibility of inconsistent data in the event of a system crash.⁴⁸

Many of today's hard disk drives maintain an independent high-speed buffer cache (often called an on-board cache) of several megabytes.⁴⁹ If requested data is stored in the on-board cache, the hard drive can deliver the data at or near the speed of main memory. Additionally, some hard disk interface controllers (e.g., SCSI and RAID controllers) maintain their own buffer caches separate from main memory. When requested data is not located in the main memory buffer cache, on-board buffer caches improve I/O performance by servicing requests without performing relatively slow mechanical operations.⁵⁰ However, one study showed that disks that include on-board caches typically use replacement strategies that adapt poorly to random request patterns, leading to suboptimal performance.⁵¹ It is also likely that on-board buffer caches and main memory buffer caches will contain the same data, which leads to inefficient resource utilization.

Self Review

1. What are the trade-offs when selecting the size of the system's disk cache buffer?
2. State when write-back caching and write-through caching are appropriate, and why.

Ans: 1) A small disk cache buffer allows enough memory for active processes, but is likely to contain only a small portion of requested data, so many requests must be serviced using the disk. A small cache buffer also limits the number of write requests that can be buffered simultaneously, which means that data will need to be flushed frequently. A large cache buffer reduces the number of I/O operations the system performs, but can reduce the memory available to store process instructions and data, which could lead to thrashing. 2) Write-back caching enables the system to batch modified data to reduce the number of disk requests, but does not prevent loss of buffered data in the event of a power or system failure. Thus, write-back caching is appropriate for systems where performance is more important than reliability (e.g., some supercomputers). Because write-through caching immediately sends modified data to disk, it is appropriate for systems that cannot tolerate loss of data (e.g., database systems).

12.9 Other Disk Performance Techniques

We have considered optimizing the performance of rotational storage devices using scheduling policies and optimizing system architecture. In this section we analyze several other disk performance optimization techniques.

As files and records are added and deleted, a disk's data tends to become dispersed throughout the disk, or **fragmented**. Even sequential files, which one would expect to exhibit low access latency, become severely fragmented, which increases access time. Many operating systems provide **defragmentation** (or **disk reorganization**) programs that can be used periodically to reorganize files. This allows the consecutive records of sequential files to be placed contiguously on the disk. Systems that use noncircular variations of the SCAN strategy tend to visit the midrange cylinders more frequently than the outermost ones. For these systems, frequently referenced items can be placed on the midrange tracks to reduce average access times.

Also, operating systems can place files that are likely to be modified near free space to reduce future fragmentation—as the files grow, new data can be placed in the adjacent free space instead of elsewhere on disk. Some operating systems allow users to **partition** a disk into separate areas. Files are then restricted to these partitions, so fragmentation is reduced.⁵² However, partitions can lead to wasted memory similar to internal fragmentation in paged virtual memory systems.

Some systems use **data compression** techniques to reduce the amount of space required by information on disk (see the Operating Systems Thinking feature, Compression and Decompression). Data compression decreases the size of a record by replacing common bit patterns with shorter ones. Therefore, compressed data consumes less space without losing information. This can ultimately reduce the number of seeks, latency times and transmission times. However, it can require substantial processor time to compress the data and later to decompress it to make it available to applications.

Systems that need to access certain information quickly benefit from placing multiple copies of that data at different positions on the disk (see the Operating

Systems Thinking feature, Redundancy). This can substantially reduce seek and rotation times, but the redundant copies can consume a significant portion of the disk. This technique is more useful for read-only data, or for data that changes nominally. Frequent changes degrade the performance of the scheme, because each of the copies must be updated regularly; another danger is that a system crash could leave the multiple copies in an inconsistent state.

In a multiple-disk system in which only one disk request may be serviced at a time, performance can be improved by replicating frequently referenced data on separate drives, which enables higher throughput. Some RAID techniques, discussed in Section 12.10, implement this concept.

Additionally, record **blocking** can yield significant performance improvements. When contiguous records are read or written as a single block, only a single seek is required; if these records are read or written individually, then one seek per record could be required.

Systems that monitor disk access attempts can try to keep frequently accessed data in a favorable position in the memory hierarchy (i.e., in main or cache memory) while transferring infrequently referenced data to slower storage (such as a hard disk or compact disk). This yields an overall performance improvement, but it might provide poor response to users of infrequently accessed data. This is unacceptable if the infrequently accessed data belongs to high-priority applications. It might also be helpful to dedicate a disk to a single high-performance application.

In many environments, there are often short periods when there are no waiting requests for the disk to service and the disk arm sits idle, waiting for the next



Operating Systems Thinking

Compression and Decompression

An excellent use of abundant, cheap processor power is compression and decompression of data. On hard disks, this helps reduce the space required to store files. It is also important in network transmissions. The world's networking infrastructure bandwidths tend to increase much slower than processor power does. The capital investment in

networking infrastructure, especially transmission lines, is enormous. It simply is not feasible to keep upgrading the entire world's networking infrastructure every time a faster transmission technology becomes available. This means that the ratio of processor speeds to transmission speeds will tend to increase over time, with processor power remaining rela-

tively cheap compared to transmission bandwidth. Compression enables messages to be shrunk for transmission over relatively slow networks, essentially increasing network throughput. Compressed messages must be decompressed at the receiving end to be usable. Operating systems will increasingly include compression and decompression mechanisms.

request.⁵³ If the disk arm is currently at one edge of the disk, it is likely that a long seek will be required to process the next request. By contrast, if the disk arm is in the center of the disk, or at some **hot spot** of disk activity, the average seek time will be less. Moving the disk arm to a location that will minimize the next seek is known as **disk arm anticipation**.⁵⁴

Disk arm anticipation can be useful in environments where the disk request patterns of processes exhibit locality. For best results, the arm should move to the hot spot of disk activity (rather than simply the center of the disk). When processes issue requests to sequential locations on disk, however, moving the head to the center of the disk after servicing each request can lead to excessive seek activity. In this case, disk arm anticipation might degrade performance.⁵⁵

Self Review

1. How do defragmentation and disk reorganization improve performance?
2. Why might data compression techniques become more practical in the future?
3. Under what conditions is it inappropriate to move the read/write head to the center of the disk during idle periods?
4. Why does multiprogramming complicate efforts to anticipate the next request?

Ans: 1) Defragmentation places file data in contiguous blocks on disk, which improves access times by reducing seek activity when accessing sequential data. Disk reorganization places frequently or heavily used data in favorable locations on disk (e.g., midrange tracks for noncircular scheduling strategies) to reduce average seek times. 2) There is a growing gap between processor speed and disk speed. The reduced access time due to data compression



Operating Systems Thinking

Redundancy

We will see many examples of redundancy being employed in operating systems for a variety of reasons. A common use of redundancy is creating backups to ensure that if one copy of information is lost, it can be restored. A multiprocessing system can have a pool of identical processors available to assign to processes and threads as needed. Such redundancy has several advantages. Although the system could still

function with only a single processor, having the extra processors yields better performance because the processors can all work in parallel. It is also effective for fault tolerance—if one processor fails, the system can continue operating. RAID reduces access times to data on disks by placing redundant copies of that data on separate disks that may function in parallel. We can also place redundant copies of the data on differ-

ent regions of the same disk, so that we can minimize the movement of the read/write head and the amount of rotational movement of the disk before the data becomes accessible, thus increasing performance. Redundancy, of course, has its price. The resources costs money and the hardware and the software to support them can become more complex. This is yet another example of trade-offs in operating systems.

might outweigh the overhead incurred by compressing and decompressing data. **3)** If programs generally exhibit spatial locality, and the area of locality is not at the center of the disk, then moving the head to the center of the disk during each idle period requires a wasteful seek to return to the disk's hot spot when requests resume. **4)** A multiprogrammed system can service requests from multiple concurrent processes, which may lead to several hot spots on the disk. In this case, it is difficult to determine which, if any, hot spot the read/write head should move to when it is idle.

12.10 Redundant Arrays of Independent Disk (RAID)

RAID (Redundant Arrays of Independent Disks) is a family of techniques that use multiple disks (called an array of disks) that are organized to provide high performance and/or reliability. RAID was originally proposed by Patterson, Gibson and Katz; in their paper, the "I" in RAID denotes "Inexpensive," but this has been changed to "Independent" because many RAID systems employ expensive, high-performance disks.⁵⁶

Patterson et al. recognized that processor speed, main memory size and secondary storage size were increasing rapidly while I/O transfer rates (particularly in hard disks) were growing at a much slower pace. Computer systems were becoming increasingly I/O bound—they could not service I/O requests as quickly as they were generated, and could not transfer data as quickly as it was capable of being consumed. To improve throughput and transfer rates, the authors recommended creating arrays of disks that could be accessed simultaneously.⁵⁷

12.10.1 RAID Overview

In their original paper, Patterson et al. proposed five different organizations, or **levels**, of disk arrays.⁵⁸ Each RAID level is characterized by **data striping** and **redundancy**. Data striping entails dividing storage into fixed-size blocks called **strips**. Contiguous strips of a file are typically placed on separate disks so that requests for file data can be serviced using multiple disks at once, which improves access times. A **stripe** consists of the set of strips at the same location on each disk in the array. In Fig. 12.16, a portion of a file is divided into four strips of identical length, each on a different disk in the array. Because each strip is placed at the same location on each disk, the four strips form one stripe. Striping distributes a system's data across multiple disks, which enables higher throughput than a single-disk system because data can be accessed from multiple disks simultaneously.

When selecting a strip size, the system designer should consider the average size of disk requests. Smaller strip sizes, also called **fine-grained strips**, tend to spread file data across several disks. Fine-grained strips can reduce each request's access time and increase transfer rates, because multiple disks simultaneously retrieve portions of the requested data. While these disks service a request, they cannot be used to service other requests in the system's request queue.

Large strip sizes, also called **coarse-grained strips**, enable some files to fit entirely within one strip. In this case, some requests can be serviced by only a por-

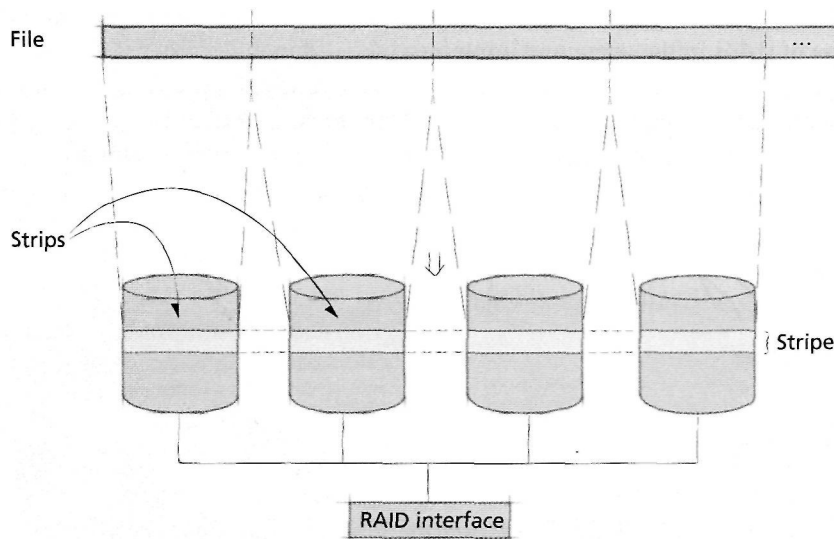


Figure 12.16 | Strips and stripe created from a single file in RAID systems.

tion of the disks in the array, so it is more likely that multiple requests can be serviced simultaneously. However, smaller requests are serviced by one disk at a time, which reduces transfer rates for individual requests compared to fine-grained strips.

Systems such as Web servers and databases, which typically access multiple small records simultaneously, benefit from coarse-grained strips, because several I/O operations can occur concurrently. Systems such as supercomputers that require fast access to a small number of records benefit from fine-grained strips, which provide high transfer rates for individual requests.⁵⁹

The increased transfer rate provided by RAID systems have a price. As the number of disks in the array increases, so does the likelihood of disk failure. For example, if the **mean-time-to-failure (MTTF)** of a single disk is 200,000 hours (roughly 23 years), the MTTF for a 100-disk array would be 2,000 hours (about three months).⁶⁰ If one disk in an array fails, any files containing strips on that disk are lost. In business-critical and mission-critical systems, this loss of data could be catastrophic (see the Operating Systems Thinking feature, Mission-Critical Systems). As a result, most RAID systems store information that enables the system to recover from errors, a technique called redundancy. RAID systems use redundancy to provide fault tolerance (i.e., to sustain failures without loss of data; see the Operating Systems Thinking feature, Fault Tolerance).

A straightforward way to provide redundancy is **disk mirroring**, a technique that places each unique data item on two disks. A drawback to mirroring is that only half of the storage capacity of the array can be used to store unique data. As we discuss in the sections that follow, some RAID levels employ a more sophisticated approach that reduces this overhead.⁶¹

To realize the improved performance of striped disks with redundancy, the system must efficiently divide files into strips, form files from strips, determine the locations of strips in the array and implement the redundancy scheme. Using a general-purpose processor for this purpose can significantly degrade the performance of processes competing for processor time. Thus, many RAID systems contain spe-



Operating Systems Thinking

Mission-Critical Systems

There are systems where the cost of failure is so high that enormous investment must be made in resources to ensure that the systems do not fail and perform well enough to meet their requirements. Such systems are often called mission-critical or business-critical systems. If an air traffic control system were to fail,

human lives could be lost. If the New York Stock Exchange systems were to fail, the world's stock markets could crash. Thus, operating systems must be designed to meet the unique reliability and performance requirements of particular applications systems. In some cases, the operating system must make extraordinary resource

commitments to particular applications, far greater than what general-purpose applications would normally receive. In the Linux and Windows XP case studies, we will see, for example, that real-time applications can be given special treatment.



Operating Systems Thinking

Fault Tolerance

Computer systems are employed in many key aspects of our personal and professional lives. We depend on computers not to fail. Fault tolerant systems are designed to keep operating, even in the presence of problems that might typically cause system failure. Redundancy is a popular means of achieving fault toler-

ance—if a component fails, another "equal" component takes over. It is costly to design fault tolerance into a system, but the costs can be small compared to the costs of system failure, especially in business-critical or mission-critical systems. We discuss issues of fault tolerance in many parts of the book, with par-

ticular emphasis on these techniques in our treatments of RAID, multiprocessing in Chapter 15, computer networking in Chapter 16, distributed systems in Chapter 17, distributed file systems in Chapter 18 and computer security in Chapter 19.

cial-purpose hardware called a **RAID controller** to perform such operations quickly. **RAID** controllers also simplify **RAID** implementation by enabling the operating system simply to pass read and write requests to the **RAID** controller, which then performs striping and maintains redundant information as necessary. However, RAID controllers can significantly increase the cost of a **RAID** system.

A systems designer choosing to adopt a **RAID** system must balance cost, performance and reliability. Typically, improving one characteristic worsens the other two. For example, to reduce the cost of the **RAID** system, one might reduce the number of disks in the array. A smaller number of disks often reduces performance by limiting transfer rates and can reduce reliability by limiting the array's capacity to store redundant information.

SelfReview

1. Explain when fine-grained and coarse-grained strips are appropriate in RAID systems.
2. Why should RAID systems provide some degree of fault tolerance?

Ans: 1) Fine-grained strips are appropriate for systems requiring individual I/O operations to be performed quickly. Coarse-grained strips are appropriate for systems that must fulfill many I/O requests simultaneously. 2) The greater the number of disks in an array, the more likely it is that one of them will fail. Thus, RAID systems should provide additional fault tolerance.

12.10.2 Level 0 (Striping)

RAID level 0 uses a striped disk array with no redundancy. Figure 12.17 illustrates a RAID 0 configuration containing four striped disks. If an application requests to read data stored in strips A and B of the array data from both strips can be read simultaneously, because they are on separate disks (D₁ and D₂). Further, the **RAID** controller can simultaneously service a write request to strip K, located on disk D₃.

RAID level 0 was **not** one of the five original levels of **RAID** and is not considered a "true" **RAID** level because it does not provide fault tolerance. If one of the disks fails, all the data in the array that depend on the failed disk are lost. Depending on the array's strip size, all data stored in the array could become unusable with the loss of a single disk.⁶²

RAID level 0 is simple to implement and does not incur storage overhead to provide fault tolerance. Further, a **RAID** 0 system with n disks performs reads and writes at a rate up to n times greater than that of a single disk. **RAID** 0 systems are appropriate for systems where high performance and low cost are more important than reliability.^{63, 64}

SelfReview

1. Why is RAID level 0 not considered a "true" RAID level?
2. What are the benefits of RAID level 0?

Ans: 1) RAID level 0 does not provide fault tolerance through redundancy. 2) RAID level 0 provides high transfer rates, is simple to implement and does not incur storage overhead to provide fault tolerance.

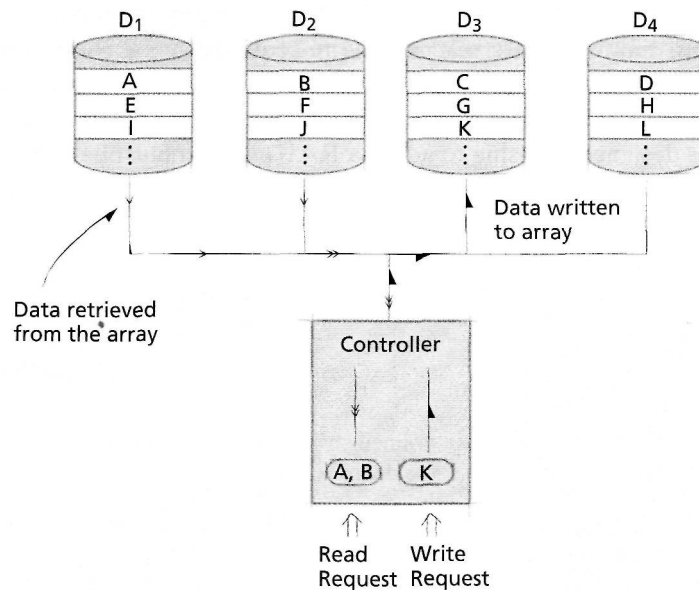


Figure 12.17 | RAID level 0 (striping).

12.10.3 Level 1 (Mirroring)

RAID level 1 employs disk **mirroring** (also called **shadowing**) to provide redundancy, so each disk in the array is duplicated. Stripes are not implemented in level 1, reducing both hardware complexity and system performance. Figure 12.18 displays a RAID 1 array. Notice that disks D₁ and D₂ contain the same data, and disks D₃ and D₄ contain the same data. Level 1 arrays permit multiple I/O operations to be serviced simultaneously. For example, in Fig. 12.18, requests to read data stored on blocks A and B can be serviced concurrently by each disk of the mirrored pair, D₁ and D₂. To ensure consistency, modified data must be written to a pair of mirrored disks, so multiple write requests to the same mirrored pair must be serviced one at a time. Note that in Fig. 12.18, read requests for blocks A and B can be serviced by disks D₁ and D₂ while a write request to block I is serviced by disks D₃ and D₄.

Although RAID level 1 provides the highest degree of fault tolerance of any RAID level, only half of the array's disk capacity can be used to store unique data. Thus, the cost per unit storage in a RAID 1 array is twice that in a RAID 0 array. Because each block of data is stored on a pair of disks, the system can sustain multiple disk failures without loss of data. For example, if disk D₃ fails, the system can continue to operate using the mirrored data on D₄. A new disk can subsequently be installed to replace D₃. Recovering and rebuilding data from a failed disk, called **data regeneration**, entails copying the redundant data from the mirrored disk. However, if both disks in a mirrored pair fail, their data is unrecoverable.⁶⁵

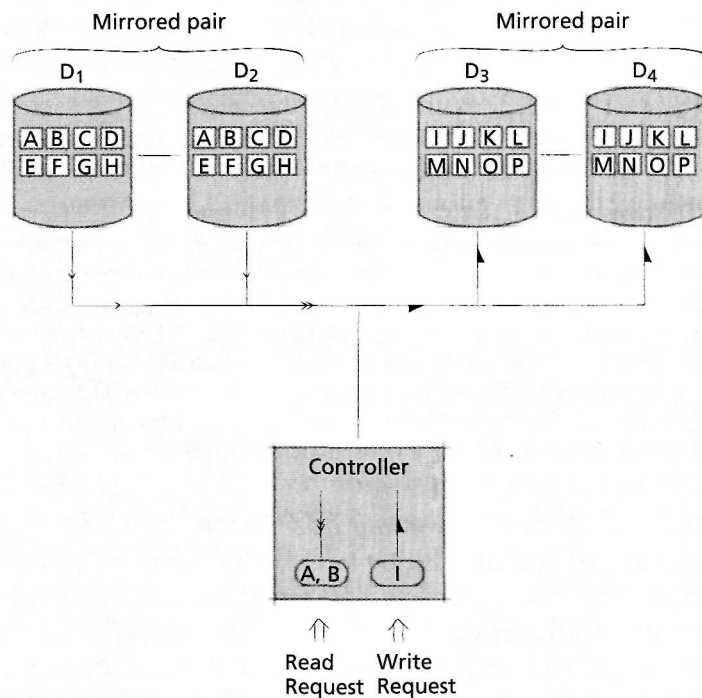


Figure 12.18 | RAID level 1 (mirroring).

Some RAID systems contain "spare" disks (also called **hot spare disks** or **online spares**) that can replace failed disks, much like a spare tire for an automobile. Some systems also feature hot swappable disk drives that can be replaced while the system is online, which allows data regeneration to occur as the system runs—necessary in high-availability environments such as online transaction-processing systems.⁶⁶

Level 1 arrays are characterized by the following advantages and disadvantages:

- High storage overhead—only half of the array's storage capacity can be used to store unique data.
- Fast average read transfer rates—two different read requests to data stored in the same mirrored pair can be serviced simultaneously.
- Slower average write transfer rates—multiple write requests to a mirrored pair must be performed one at a time. However, write requests to different mirrored pairs can be performed simultaneously.
- High fault tolerance—RAID level 1 can sustain multiple drive failures without loss of data or availability. It provides the highest degree of fault tolerance among the most popular RAID levels.
- High cost—the storage overhead increases cost per unit capacity.

RAID 1 is best suited for environments in which reliability is a higher priority than cost or performance.⁶⁷

Self Review

1. Compare and contrast RAID 0 and RAID 1 in terms of cost, performance and reliability.
2. Why is RAID 1 appropriate for business- and mission-critical systems?

Ans: **1)** RAID 0 costs less than RAID 1 when capacity is held fixed. Read and write performance is dependent on system load and strip sizes, but both RAID 0 and RAID 1 can service read requests for data on different disks simultaneously. RAID 0 can service write requests for data on different disks simultaneously; RAID 1 can perform write requests simultaneously only if they are to data stored on different mirrored pairs. RAID 0 is much more likely to fail than a single disk; RAID 1 is much less likely to fail than a single disk. **2)** In these systems, performance, reliability and availability are more important than cost. RAID 1 offers high performance for read requests and extremely high reliability, and it can continue to operate without loss of data if one or more disks fail (but at reduced performance levels).

12.10.4 Level 2 (Bit-Level Hamming ECC Parity)

RAID level 2 arrays are striped at the bit level, so each strip stores one bit. Level 2 arrays are not mirrored, which reduces the storage overhead incurred by level 1 arrays. When a disk in a RAID system fails, it is analogous to a portion of a message that has been corrupted during data transfer, such as during network transmission. The problem of unreliable transmission has been well researched and several solutions have been proposed (for examples of solutions to the problem of unreliable transmission, see www.eccpage.com). RAID level 2 borrows from a technique that is commonly implemented in memory modules called **Hamming error-correcting codes (Hamming ECCs)**, which uses parity bits to check for errors in data transmitted from disks and to correct them, if possible.⁶⁸

Parity bits can be calculated as follows. Consider an array in which each stripe stores four bits. When the RAID system receives a request to write the data 0110 to a stripe, it determines the **parity** of the sum of the bits, i.e., whether the sum is odd or even. In this case, the parity sum (2) is even, so the system writes the stripe to the corresponding four data disks of the array and a 0, representing even parity, to a parity disk. [Note: Although data is striped at the bit level, most requests correspond to data in multiple stripes that can be written at once to improve performance.] RAID systems generally place parity bits on a separate disk, so that reads and writes to each stripe can be performed by all disks simultaneously. When the stripe is next accessed, the system reads the stripe and its corresponding parity bit, calculates the stripe's parity and compares it to the value read from the parity disk. If a disk error causes the third bit of the stripe to change from 1 to 0 (i.e., the stripe stores 0100), the stripe will have odd parity, and the system can detect the error by comparing the parity to the value stored on the parity disk.

A limitation of this particular form of parity is that it cannot detect an even number of errors. For example, if both the second and third bits change from 1 to 0

(i.e., the stripe stores 0000), the stripe will still have even parity. Further, this technique does not enable the system to determine which, if any, bits have errors.

Hamming error-correcting codes (Hamming ECCs) use a more sophisticated approach to enable the system to detect up to two errors, correct up to one error and determine the location of the error in a stripe (the algorithm behind Hamming ECC parity generation is beyond the scope of this book; it is described at www2.rad.com/networks/1994/err_con/hamming.htm). The size of Hamming ECC codes, and thus the number of parity disks, increases according to the logarithm of the number of data disks. For example, an array of 10 data disks requires four parity disks; an array of 25 data disks requires five parity disks.⁶⁹ Thus, level 2 arrays containing a large number of disks incur significantly less storage overhead than level 1 arrays.

Figure 12.19 illustrates a level 2 array containing four data disks and three parity disks. Stripe A is composed of the strips A0-A3, which occupy the first strips on the data disks D1-D4. Stripe A's corresponding Hamming ECC is composed of the bits A_x - A_z , which occupy the first strips on the parity disks P1-P3.

Consider a write request to store data in strips A_0 , A_1 , A_2 , A_3 , B_0 and B_1 . In this case, because the Hamming ECC code is computed for each write, the controller must write stripes A and B, and the Hamming ECC code for each stripe. Even

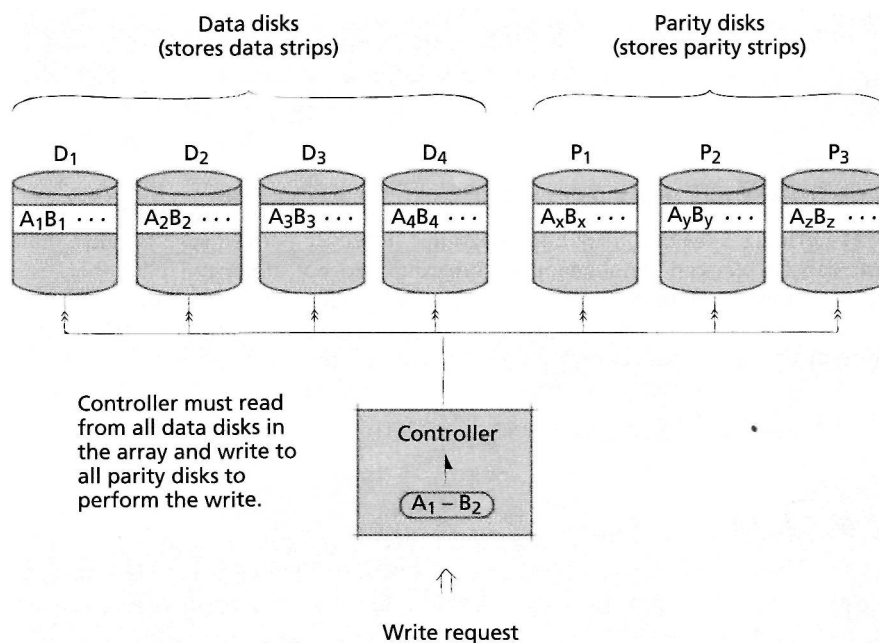


Figure 12.19 | RAID level 2 (bit-level ECC parity).

though only two of the four bits in stripe B are modified, the entire B stripe must be read and its Hamming ECC code calculated and written to disk. This process, which degrades performance because the system must access the array twice for each write, is called a **read-modify-write cycle**.

Although the ECC disks reduce the overhead associated with redundancy when compared to mirroring, they can degrade performance because multiple requests cannot be serviced simultaneously. Each read request requires the array to access all disks, compute the ECC, and compare it with the value retrieved from the parity disks. Some systems reduce this bottleneck by computing parity information only for write requests. However, the system must access all disks to maintain parity information when performing a write, meaning that write requests must be serviced one at a time.

One way to increase the maximum number of concurrent reads and writes using a RAID level 2 array is to divide the system's disks into several small RAID level 2 arrays. Unfortunately, this technique increases storage overhead, because smaller groups require a larger ratio of parity disks to data disks. For example, a 32-data-disk level 2 array requires six parity disks. If the system were divided into eight groups of four data disks, the resulting arrays would require 24 parity disks.⁷⁰

The primary reason that RAID 2 is not implemented in today's systems is that modern disk drives maintain Hamming ECCs or comparable fault-tolerance mechanisms transparently in hardware. Almost all current SCSI and IDE drives contain built-in error detection and disk monitoring.⁷¹

Self Review

1. How does dividing a large RAID level 2 array into subgroups affect cost, performance and reliability?
2. Why is RAID level 2 rarely implemented?

Ans: 1) Dividing a level 2 array into subgroups improves performance because multiple requests can be serviced simultaneously. Subgroups do not affect reliability, because the same error-checking routines are performed for each subgroup. As a whole, the subgroups require more parity disks than a single large array, which increases cost. 2) Because modern hardware devices have built-in error detection and disk monitoring that perform the same checks as level 2 arrays, designers seek a lower-cost and higher-performance alternative to providing fault tolerance.

12.10.5 level 3 (Bit-level XOR ECC Parity)

RAID level 3 stripes data at the bit or byte level. Instead of using Hamming ECC for parity generation, RAID 3 uses **XOR (exclusive or) error-correcting codes (XOR ECCs)**. The XOR ECC algorithm is much simpler than the Hamming ECC. It is called XOR ECC because it uses the logical XOR operation:

$$\begin{aligned}(a \text{ XOR } b) &= 0 \text{ when } a \text{ and } b \text{ are both zero or both one;} \\ &= 1 \text{ otherwise.}\end{aligned}$$

It follows that $(a \text{ XOR } (b \text{ XOR } c)) = 0$ only when an even number of the arguments are 1 or 0. RAID level 3 exploits this fact to perform nested XOR operations on each byte to generate its XOR ECC. For example, consider stripe A in Fig. 12.20. Let $A_0 = 1$, $A_1 = 0$, $A_2 = 0$ and $A_3 = 1$; or, more succinctly, $A = 1001$. The system uses nested XOR operations to compute A's parity. If the number of 1's or 0's is even, the parity bit, A_p , is 0; otherwise, A_p is 1. In this case there are two 1's, so $A_p = 0$. If the system stripes at the byte level, the XOR ECC is calculated for each bit, so each parity strip stores 8 bits.

XOR ECC uses only one disk to hold parity information, regardless of the size of the array. Note that, unlike Hamming ECCs, XOR ECCs do not enable the system to detect which bit contains erroneous data. This is acceptable, because most parity errors in RAID systems result from the failure of an entire disk, which is easy to detect. For example, assume that disk D_2 has failed (Fig. 12.20). In this case, when the system attempts to read stripe A, it finds $A = 1x01$ (where x is the unknown bit) and $A_p = 0$. Because $A_p = 0$, stripe A must have stored an even number of 1's. Thus, the system can determine that $A_2 = 0$ and regenerate the lost data. This technique can be applied to recover from any single data disk failure; if the parity disk fails, recovery entails recomputing the parity from the data disks.⁷²

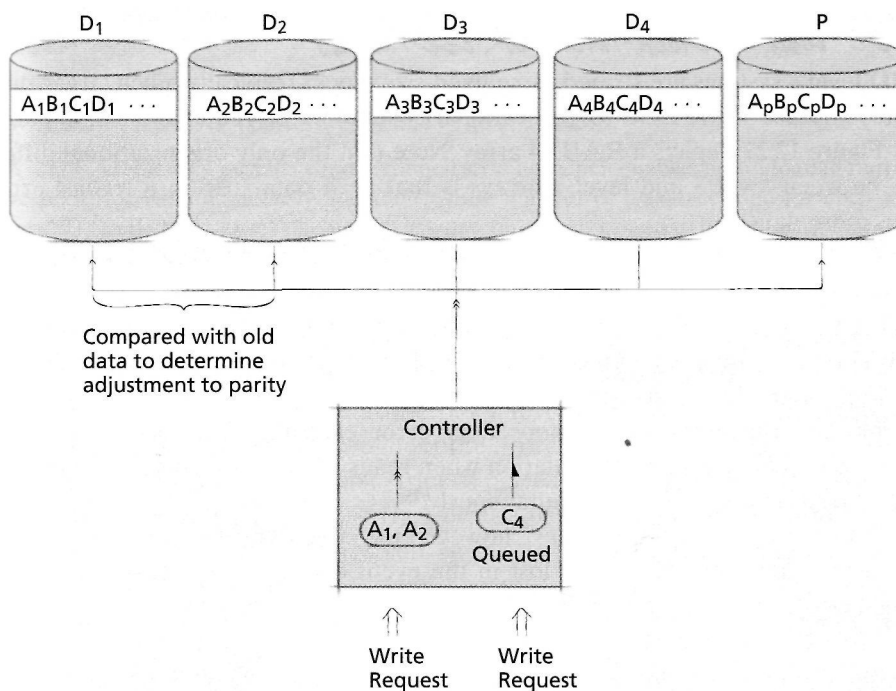


Figure 12.20 | RAID level 3 (bit-level, single parity disk).

Due to fine-grained striping, most reads require access to the entire array. Further, due to parity generation, only one write can be performed at a time. Similar to RAID level 2, this yields high transfer rates when reading and writing large files, but, in general, only one request can be serviced at a time.⁷³ The primary advantage of RAID level 3 is that it is easy to implement, offers reliability similar to RAID level 2, and entails significantly lower storage overhead than RAID level 2.⁷⁴

Self Review

1. Compare and contrast RAID levels 2 and 3.
2. Why is XOR ECC parity, instead of Hamming ECC parity, sufficient for most RAID systems?

Ans: 1) Both RAID levels 2 and 3 provide fault tolerance using parity to reduce the overhead of mirroring, both use dedicated parity disks and, in general, both cannot service multiple requests simultaneously. RAID level 2 incurs greater storage overhead than level 3. **2)** Although XOR ECCs cannot determine which disk has failed, the system can easily determine this, because failed disks do not respond to requests. Therefore, the system can regenerate one failed disk using a single parity disk.

12.10.6 Level 4 (Block-Level XOR ECC Parity)

RAID level 4 systems are striped using fixed-size blocks (typically much larger than a byte) and use XOR ECC to generate parity data, which requires a single parity disk. Figure 12.21 depicts a RAID 4 array. Note that the only organizational difference between level 3 and level 4 arrays is that each data strip in a level 4 array stores more data.

In RAID levels 2 and 3, each request's data is typically stored on each data disk in the array. However, because RAID level 4 permits coarse-grained strips, it is possible that requested data might be stored on a small fraction of the disks in the array. Thus, if parity is not determined for each read, the system can potentially service multiple read requests simultaneously. Because the ECC codes are used primarily for data regeneration as opposed to error checking and correcting, many systems eliminate the parity calculation when reads are performed so that multiple read requests can be serviced simultaneously.⁷⁵

When servicing a write request, however, the system must update parity information to ensure that no data is lost in the event of a disk failure. When coarse-grained strips are employed, write requests rarely modify data on each disk in the array. Again, accessing each disk in the array to compute parity information can lead to substantial overhead. Fortunately, the system can calculate the new parity bit, A_p' , simply by using the data block before modification, A_d , the modified data block, A_d' , and the corresponding parity block on disk, A_p :

$$A_p' = (A_d \text{ XOR } A_d') \text{ XOR } A_p.^{76}$$

You will be asked to prove this relation in the exercises. This technique eliminates wasteful disk I/O operations, because writes do not require access to the entire array. However, because each write request must update the parity disk, write requests must be performed one at a time, creating a write bottleneck. As we discuss in the next section, RAID level 5 removes the write bottleneck, so RAID level 4 is rarely implemented.^{77-78,79,80}

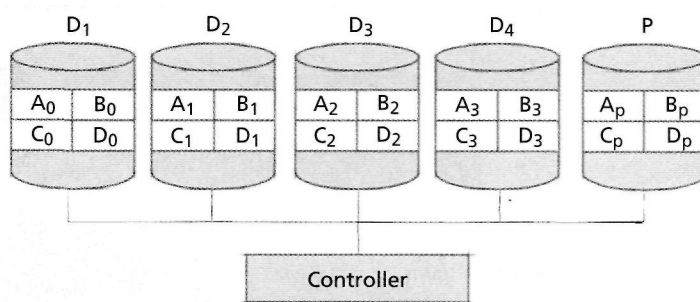


Figure 12.21 | RAID level 4 (block-level parity).

Self Review

1. How does RAID level 4 offer higher performance than RAID level 3?
2. Why can RAID level 4 perform only one write at a time?

Ans: 1) RAID level 4 permits coarse-grained striping, which enables multiple read requests to be performed simultaneously and reduces the number of disks accessed for each write request. 2) RAID level 4 uses one parity disk that must be accessed to update parity information each time data in the array is modified.

12.10.7 Level 5 (Block-Level Distributed XOR ECC Parity)

RAID level 5 arrays are striped at the block level and use XOR ECC parity, but parity blocks are distributed throughout the array of disks (Fig. 12.22).⁸¹ Note that in Fig. 12.22, the parity for the first stripe is placed on disk D5, while the parity for the second stripe is placed on disk D4. Because parity blocks are distributed across many disks, multiple parity strips can be accessed simultaneously, removing the write bottleneck for many requests. For example, consider how RAID level 5 services write requests to strips A₁ and C₂. The controller accesses disk D₂ to write strip A₁ and disk D₅ to update the corresponding parity block, A_p. The controller can simultaneously access disk D₄ to write strip C₂ and access disk D₃ to update its corresponding parity block, C_p.

Although RAID level 5 improves write performance by distributing parity, level 5 arrays must still perform a read-modify-write cycle for each write request,

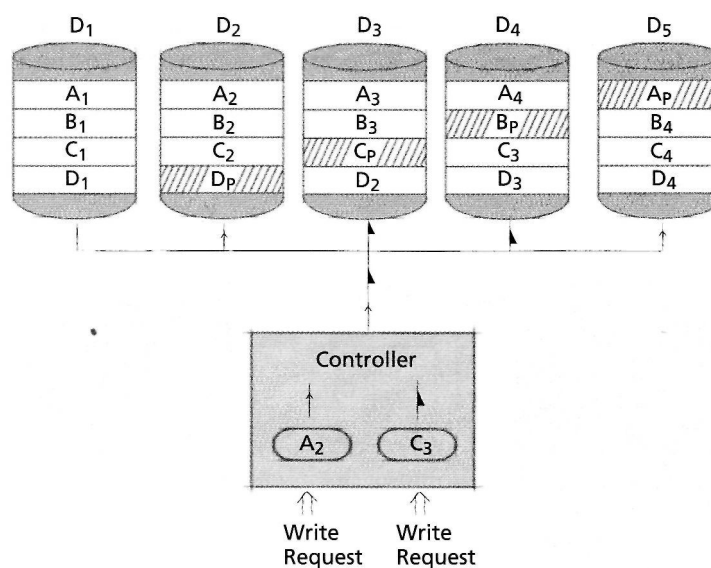


Figure 12.22 | RAID level 5 (block-level distributed parity).

requiring at least four I/O operations to service each write request. If a system consistently writes small amounts of data, the number of costly I/O operations can significantly degrade performance.⁸²

Several methods have been developed to address this issue. Caching recently accessed data and parity blocks can reduce the number of I/O operations in the read-modify-write cycle. **Parity logging** can improve RAID level 5 performance by storing the difference between the old parity and new parity (called the **update image**) in memory instead of performing a read-modify-write cycle. Because a single update image can store parity information corresponding to multiple write requests, the system can reduce I/O overhead by performing a single update to the parity block in the array after several writes have been performed.⁸³ A similar performance-enhancing technique is called **AFRAID (A Frequently Redundant Array of Independent Disks)**. Instead of performing the read-modify-write cycle with every write, parity data generation is deferred to times when system load is light. This can greatly improve performance in environments characterized by intermittent bursts of requests generating small writes.⁸⁴

Although RAID level 5 increases performance relative to RAID levels 2-4, it is complex to implement, which increases its cost. Also, because parity is distributed throughout the array, data regeneration is more complicated than in other RAID levels.⁸⁵ Despite its limitations, RAID level 5 is frequently adopted due to its effective balance of performance, cost and reliability. Level 5 arrays are considered general-purpose arrays and are often found in file and application servers, enterprise resource planning and other business systems.⁸⁶ Figure 12.23 compares the six levels of RAID presented in this chapter.

<i>RAID level</i>	<i>Read Concurrency</i>	<i>Write Concurrency</i>	<i>Redundancy</i>	<i>Striping level</i>
0	Yes	Yes	None	Block
1	Yes	No	Mirroring	None
2	No	No	Hamming ECC parity	Bit
3	No	No	XOR ECC parity	Bit/byte
4	Yes	No	XOR ECC parity	Block
5	Yes	Yes	Distributed XOR ECC parity	Block

Figure 12.23 | Comparison of RAID levels 0-5.

Other RAID levels

Many other RAID levels exist, each borrowing from one or more of the RAID levels we discussed in the preceding sections.⁸⁷ Unfortunately, there is no standard naming convention for RAID levels, which often leads to confusing or misleading names. Some of the more notable RAID levels include

- RAID level 6—extends RAID 5 by distributing two parity blocks per stripe for higher reliability upon disk failure.⁸⁸
- RAID level 0+1—a set of striped disks (level 0) whose image is mirrored to a second set of disks (level 1).
- RAID level 10—a set of mirrored data (level 1) that is striped across another set of disks (level 0), requiring a minimum of four disks.

Other RAID arrays that have been developed include levels 0+3, 0+5, 50,1+5, 51 and 53.⁸⁹ RAID level 7, which is a high-performance, high-cost solution, is a proprietary technique whose name is trademarked by the Storage Computer Corporation.⁹⁰

Self Review

1. How does RAID level 5 address the write bottleneck in RAID level 4?
2. What techniques have been implemented to improve performance in RAID systems that receive many requests for small writes?

Ans: 1) RAID level 5 distributes parity information across all disks in the array so that multiple write operations can be performed simultaneously. 2) Caching recently accessed data improves write performance by reducing the number of I/O operations generated by read-modify-write cycles. Delaying the update of parity information improves performance by reducing the number of disks accessed to perform each write operation.

Web Resources

www.disk-tape-data-recovery.com/storage-history.htm

Provides a short history of persistent data storage media.

www.almaden.ibm.com/sst/storage/index.shtml

Describes physical limitations of storage technology and promising fields of research IBM has pursued to solve them.

colossalstorage.net/colossal.html

Provides links to sites describing evolving storage technologies.

www.cose.brocku.ca/~cspress/HelloWorld/1999/03-mar/disk_scheduling_algorithms.html

Contains an article from *Hello World*, a magazine for computer science students; the article overviews disk scheduling algorithms.

www.peguide.com/ref/hdd/index.htm

Discusses hard disk characteristics.

www.microsoft.com/windows2000/techinfo/administration/fileandprint/defrag.asp

Describes how to use the disk defragmenting utilities in Windows 2000.

www.pcguide.com/ref/hdd/perf/raid/

Provides an in-depth introduction to RAID.

www.acne.com/04_00.html

Provides a summary of most RAID levels.

linas.org/linux/raid.html

Contains articles about RAID and its implementation in Linux systems.

www.pdl.emu.edu/RAID/

Overviews RAID and provides links to current RAID research.

Summary

As processor and main memory speeds increase more rapidly than those of secondary storage devices, optimizing disk performance has become important to realizing optimal performance. Magnetic storage records data by changing the direction of magnetization of regions, each representing a 1 or a 0. To access data, a current-carrying device called a read/write head hovers above the medium as it moves.

Most modern computers use hard disks as secondary storage. As the platters spin, each read-write head sketches out a circular track of data on a disk surface. All read-write heads are attached to a single disk arm (also called an actuator, boom or moving-arm assembly). When the disk arm moves the read/write heads to a new position, a different set of tracks, or cylinder, becomes accessible. The process of moving the disk arm to a new cylinder is called a seek operation. The time it takes for the head to move from its current cylinder to the one containing the data record being accessed is called the seek time. The time it takes for data to rotate from its current position to a position adjacent to the read/write head is called latency time. Then the record, which is of arbitrary size, must be made to spin by the read/write head so that the data can be read from or written to the disk. This is called transmission time.

Many processes can generate requests for reading and writing data on a disk simultaneously. Because these processes sometimes make requests faster than they can be serviced by the disk, queues build up to hold disk requests. Some early computing systems simply serviced these requests on a first-come-first-served (FCFS) basis, in which the earliest arriving request is serviced first. FCFS is a fair

method of allocating service, but when the request rate (i.e., the load) becomes heavy, FCFS results in long waiting times.

FCFS exhibits a random seek pattern in which successive requests can cause time-consuming seeks from the innermost to the outermost cylinders. To reduce the time spent seeking records, it seems reasonable to order the request queue in some other manner than FCFS. This reordering is called disk scheduling. The two most common types of scheduling are seek optimization and rotational optimization. Disk scheduling strategies often are evaluated by comparing their throughput, mean response time and variance of response times.

First-come-first-served scheduling uses a FIFO queue so that requests are serviced in the order they arrive. Although this technique incurs low execution-time overhead, it can result in low throughput due to lengthy seeks.

Shortest-seek-time-first (SSTF) scheduling next services the request that is closest to the read-write head's current cylinder (thus incurring the shortest seek time), even if that is not the first one in the queue. By reducing average seek times, SSTF achieves higher throughput rates than FCFS, and mean response times tend to be lower for moderate loads. One significant drawback is that higher variances occur on response times because of the discrimination against the outermost and innermost tracks: in the extreme, indefinite postponement of requests far from the read-write heads could occur.

The SCAN scheduling strategy reduces unfairness and variance of response times by choosing the request that requires the shortest seek distance in a preferred direction.

Thus, if the preferred direction is currently outward, the SCAN strategy chooses the shortest seek distance in the outward direction. SCAN behaves much like SSTF in terms of high throughput and good mean response time. However, because SCAN ensures that all requests in a given direction will be serviced before the requests in the opposite direction, it offers a lower variance of response times than SSTF.

The circular SCAN (C-SCAN) modification to the SCAN strategy moves the arm from the outer cylinder to the inner cylinder, servicing requests on a shortest-seek basis. When the arm has completed its inward sweep, it jumps (without servicing requests) to the outermost cylinder, then resumes its inward sweep processing requests. C-SCAN maintains high levels of throughput while further limiting variance of response times by avoiding the discrimination against the innermost and outermost cylinders. SCAN is often called the elevator algorithm.

The FSCAN and JV-Step SCAN modifications to the SCAN strategy eliminate the possibility of indefinitely postponing requests. FSCAN uses the SCAN strategy to service only those requests waiting when a particular sweep begins (the "F" stands for "freezing" the request queue at a certain time). Requests arriving during a sweep are grouped together and ordered for optimum service during the return sweep. jV-Step SCAN services the first n requests in the queue using the SCAN strategy. When the sweep is complete, the next n requests are serviced. Arriving requests are placed at the end of the request queue, which prevents requests in the current sweep from being indefinitely postponed. FSCAN and N-Step SCAN offer good performance due to high throughput, low mean response times and a lower variance of response times than SSTF and SCAN.

The LOOK variation of the SCAN strategy (also called the elevator algorithm) "looks" ahead to the end of the current sweep to determine the next request to service. If there are no more requests in the current direction, LOOK changes the preferred direction and begins the next sweep, stopping when passing a cylinder that corresponds to a request in the queue. This strategy eliminates unnecessary seek operations experienced by other variations of the SCAN strategy by preventing the read/write head from moving to the innermost or outermost cylinders unless it is servicing a request to those locations.

The circular LOOK (C-LOOK) variation of the LOOK strategy uses the same technique as C-SCAN to reduce the bias against requests located at extreme ends of the platters. When there are no more requests on a current sweep, the read/write head moves to the request closest to the outer cylinder (without servicing requests in between)

and begins the next sweep. The C-LOOK strategy is characterized by potentially lower variance of response times compared to LOOK, and by high throughput, although lower than that of LOOK.

In many environments, there are often short periods when there are no waiting requests for the disk to service and the disk arm sits idle, waiting for the next request. If the disk arm is in the center of the disk, or at some hot spot of disk activity, the average seek time will be less. Moving the disk arm to a location that will hopefully minimize the next seek is known as disk arm anticipation. It can be useful in environments where processes's disk request patterns exhibit locality and where there is typically enough time to move the disk arm between disk requests. If a request is received during anticipatory movement, moving the head to the originally specified location and then to the requested cylinder increases response time. Allowing the seek to a hot spot to be interrupted to adjust to requests provides the greatest performance boost.

Rotational optimization was used extensively with early fixed-head devices such as drums, and can improve performance in today's hard disks, which exhibit seek times and average latencies of the same order of magnitude. The shortest-latency-time-first (SLTF) strategy examines all pending requests and services the one with the shortest rotational delay first. Rotational optimization is sometimes referred to as sector queuing; requests are queued by sector position around the disk, and the nearest sectors are serviced first.

The STLF strategy works well for fixed-head disks but does not incorporate the positioning time, which is the seek time plus latency, of moving-head disks. The shortest-positioning-time-first (SPTF) strategy next services the request that requires the shortest positioning time. SPTF results in high throughput and a low mean response time, similar to SSTF, and can also indefinitely postpone requests to the innermost and outermost cylinders.

A variation of SPTF is the shortest-access-time-first (SATF) strategy, which next services the request that requires the shortest access time (i.e., positioning time plus transmission time). SATF exhibits higher throughput than SPTF, but large requests can be indefinitely postponed by a series of smaller requests. Requests to the innermost or outermost cylinders can be indefinitely postponed by requests to intermediate cylinders. SPTF and SATF can improve performance by implementing the "look-ahead" mechanisms employed by the LOOK and C-LOOK strategies.

One reason why SPTF and SATF are rarely implemented is that they require knowledge of disk characteris-

tics including latency, track-to-track seek times and relative locations of sectors. Unfortunately, many of today's hard drives hide such geometry or expose misleading geometries due to hidden error-checking data and bad sectors.

When disk storage proves to be a bottleneck, some designers recommend adding more disks to the system. This does not always solve the problem, because the bottleneck could be caused by a large request load on a relatively small number of disks. When this situation is detected, disk scheduling may be used as a means of improving performance and eliminating the bottleneck. Disk scheduling might not be useful in a batch-processing system with a relatively low degree of multiprogramming. Scheduling becomes more effective as the degree of multiprogramming increases, which increases the system load and leads to unpredictable disk request patterns.

Nonuniform request distributions are common in certain situations and their consequences have been investigated. One study determined that the vast majority of disk references are to the same cylinder as the immediately preceding reference. Sophisticated file organization techniques can cause a proliferation of requests with large seek times. Such techniques are convenient for the applications designer but can complicate implementation and degrade performance.

Many systems maintain a disk cache buffer, which is a region of main memory that the operating system reserves for disk data. In one context, the reserved memory acts as a cache, allowing processes quick access to data that would otherwise need to be fetched from disk. The reserved memory also acts as a buffer, allowing the operating system to delay writing the data to improve I/O performance. The disk cache buffer presents several challenges to operating system designers, because memory allocated to the cache buffer reduces the amount allocated to processes. Also, because memory is volatile, power or system failures can lead to inconsistencies.

Write-back caching writes buffered data to disk periodically, enabling the operating system to use a single request to batch multiple I/Os, which can improve system performance. Write-through caching writes data both to the disk cache buffer and to disk each time cached data is modified. This technique prevents the system from batching requests but reduces the possibility of inconsistent data in the event of a system crash.

Many of today's hard disk drives maintain an independent high-speed buffer cache (often called an on-board cache) of several megabytes. If data requested is stored in the on-board cache, the hard drive can deliver the data at

or near the speed of main memory. However, one study showed that disks that include on-board caches typically use replacement strategies that adapt poorly to random request patterns, leading to suboptimal performance.

Disks tend to become fragmented as files and records are added and deleted. Many operating systems provide defragmentation (or disk reorganization) programs that can be used periodically to reorganize files. This allows the consecutive records of sequential files to be placed contiguously on the disk. Also, operating systems can place files that are likely to be modified near free space to reduce future fragmentation. Some operating systems allow users to partition a disk into separate areas. Files are then restricted to these areas, so fragmentation is reduced. Some systems use data compression techniques to reduce the amount of space required by information on disk. Data compression decreases the size of a record by replacing common bit patterns with shorter ones. This can reduce seeks, latency times and transmission times, but may require substantial processor time to compress the data for storage on the disk, and to decompress the data to make it available to applications.

Systems that need to access certain information quickly benefit from placing multiple copies of that data at different positions on the disk. This can substantially reduce seek and rotation times, but the redundant copies can consume a significant portion of the disk. Also, if one copy is modified, all other copies must be modified, which can degrade performance. Additionally, record blocking can yield significant performance improvements by reading contiguous records as a single block, requiring only a single-seek. Systems that monitor disk access attempts can try to keep frequently accessed data in a favorable position in the memory hierarchy (i.e., in main or cache memory) while transferring infrequently referenced data to slower storage (such as a hard disk or compact disk).

Redundant Arrays of Independent Disks (RAID) is a family of techniques that use arrays of disks to address the problem of relatively slow disk transfer speeds. Each array organization, called a level, is characterized by data striping and redundancy. Storage is divided into fixed-size strips; contiguous strips on each disk generally store noncontiguous data or data from different files. A stripe consists of the set of strips at the same location on each disk in the array.

When selecting a strip size, the system designer should consider the average size of disk requests. Smaller strip sizes, also called fine-grained strips, tend to spread file data across several disks. Fine-grained strips can reduce each request's access time and increase transfer rates

because multiple disks simultaneously retrieve portions of the requested data. While these disks service a request, they cannot be used to service other requests in the system's request queue.

Large strip sizes, also called coarse-grained strips, enable more files to fit entirely within one strip. In this case, some requests can be serviced by only a portion of the disks in the array, so it is more likely that multiple requests can be serviced simultaneously. However, smaller requests are serviced by one disk at a time, which reduces transfer rates for individual requests.

Systems such as Web servers and databases, which typically access multiple small records simultaneously, benefit from coarse-grained strips because several I/O operations can occur concurrently; systems such as supercomputers that require fast access to a small number of records benefit from fine-grained strips, which provide high transfer rates for individual requests.

As the number of disks in the array increases, so does the likelihood that one of those disks will fail, measured by the mean-time-to-failure (MTTF). Most RAID systems maintain copies of data to help enable them to recover from errors and failures, a technique called redundancy. Disk mirroring is a redundancy technique that maintains copies of each data item on two disks. A drawback to mirroring is that only half of the storage capacity of the array can be used to store unique data.

To realize the improved performance of striped disks with redundancy, the system must efficiently divide files into strips, form files from strips, determine the locations of strips in the array and implement redundancy. Many RAID systems contain special-purpose hardware called a RAID controller to perform such operations quickly. The designer must balance cost, performance and reliability when selecting a RAID level. Typically, improving one characteristic worsens the other two.

RAID level 0 uses a striped disk array with no redundancy. Level 0 arrays are not fault tolerant; if one disk fails, all the data in the array that depend on the failed disk are lost. Depending on the array's strip size, all data stored in the array could become unusable with the loss of a single disk. Although RAID 0 is not fault tolerant, it is simple to implement, yields high transfer rates and does not incur any storage overhead.

RAID level 1 employs disk mirroring (also called shadowing) to provide redundancy, so each disk in the array is duplicated. Stripes are not implemented in level 1, reducing both hardware complexity and system performance. While this results in the highest degree of fault tol-

erance of any RAID level, only half of the array's capacity can be used to store unique data, which increases cost. For each mirrored pair, two read requests can be serviced simultaneously, but only one request can be serviced at a time during a write. Recovering and rebuilding data from a failed disk, called data regeneration, entails copying the redundant data from the mirrored disk. If both disks in a mirrored pair fail, their data is unrecoverable.

Some RAID systems contain "spare" disks (also called hot spare disks or online spares) that can replace failed disks, much like a spare tire for an automobile. Some systems also feature hot swappable disk drives that can be replaced while the system is online. In these systems, data regeneration can occur while the system continues to run, which is necessary in high-availability environments such as online transaction processing systems. RAID 1 is best suited for business-critical and mission-critical environments in which reliability has higher priority than cost or performance.

RAID level 2 arrays are striped at the bit level, so each strip stores one bit. Level 2 arrays are designed to reduce the storage overhead incurred by implementing fault tolerance using mirroring. Instead of maintaining redundant copies of each data item, RAID level 2 uses a technique of storing parity information called Hamming error-correcting codes (Hamming ECCs), which allow the system to detect up to two errors, correct up to one error and determine the location of the error in a stripe. The size of Hamming ECC codes, and thus the number of parity disks, increases in proportion to the logarithm (base 2) of the number of data disks. Thus, level 2 arrays containing a large number of disks incur significantly less storage overhead than level 1 arrays.

Although the ECC disks reduce the overhead associated with redundancy when compared to mirroring, they can degrade performance, because multiple requests cannot be serviced simultaneously. Each read request requires the array to access all disks, compute the ECC, and compare it with the value retrieved from the parity disks. Similarly, when servicing a write request, the Hamming ECC must be calculated for each stripe that is written. Further, the system must perform a read-modify-write cycle, which degrades performance because the system must access the entire array twice for each write. One way to enable concurrency using a RAID level 2 array is to divide the system's disks into several small RAID level 2 arrays, but this increases storage overhead and cost. The primary reason that RAID 2 is not implemented in today's systems is that modern disk drives perform Hamming ECCs or comparable protection transparently in hardware.

RAID level 3 stripes data at the bit or byte level. RAID 3 uses XOR (exclusive or) error-correcting codes (XOR ECCs), which use the logical XOR operation to generate parity information. XOR ECC uses only one disk to hold parity information, regardless of the size of the array. The system can use the parity bits to recover from any single disk failure; if the parity disk fails, it can be rebuilt from the array's data disks. Due to parity checking RAID level 3 reads and writes require access to the entire array. Similar to RAID level 2, this yields high transfer rates when reading and writing large files, but only one request can be serviced at a time.

RAID level 4 systems are striped using fixed-size blocks (typically larger than a byte) and use XOR ECC to generate parity data that is stored on a single parity disk. Because level 4 arrays enable coarse-grained striping, the system can potentially service multiple read requests simultaneously, if parity is not determined for each read. When servicing a write request, however, the system must update parity information to ensure that no data is lost in the event of a disk failure. This means that write requests must be performed one at a time, creating a write bottleneck.

RAID level 5 arrays are striped at the block level and use XOR ECC for parity, but parity blocks are distributed throughout the array of disks. Because parity blocks are distributed across many disks, multiple parity blocks can be accessed simultaneously, removing the write bottleneck for many requests. Although RAID level 5 improves write per-

formance by distributing parity, level 5 arrays must still perform a read-modify-write cycle for each write request, requiring at least four I/O operations to service each write request.

Caching recently accessed data and parity blocks can reduce the number of I/O operations in the read-modify-write cycle. Parity logging can improve RAID level 5 performance by storing the difference between the old parity and new parity (called the update image) in memory instead of performing a read-modify-write cycle. A Frequently Redundant Array of Independent Disks (AFRAID) improves performance by deferring parity data generation to times at which the system load is light.

Although RAID level 5 increases performance relative to RAID levels 2-4, it is complex to implement, which increases its cost. Also, because parity is distributed throughout the array, data regeneration is more complicated than in other RAID levels. Despite its limitations, RAID level 5 is frequently adopted for its effective balance of performance, cost and reliability. Level 5 arrays are considered general-purpose arrays and are often found in file and application servers, enterprise resource planning (ERP) and other business systems.

Many other RAID levels exist, each borrowing from one or more of RAID levels 0-5. Unfortunately, there is no standard naming convention for RAID levels, which often leads to confusing or misleading names.

Key Terms

A Frequently Redundant Array of Independent Disks (AFRAID) (RAID)—RAID implementation that defers parity data generation to times when the system's load is light to improve performance when a system performs many small writes.

actuator—See disk arm.

anticipatory movement—Movement of the disk arm during disk arm anticipation.

block—Fixed-size unit of data, typically much larger than a byte. Placing contiguous data records in blocks enables the system to reduce the number of I/O operations required to retrieve them.

blocking—Grouping of contiguous records into larger blocks that can be read using a single I/O operation. This technique reduces access times by retrieving many records with a single I/O operation.

boom—See disk arm.

bottleneck—Condition that occurs when a resource receives requests faster than it can process them, which can slow process execution and reduce resource utilization. Hard disks are a bottleneck in most systems.

circular LOOK (C-LOOK) disk scheduling—Disk scheduling strategy that moves the arm in one direction, servicing requests on a shortest-seek basis. When there are no more requests on a current sweep, the read-write head moves to the request closest to the cylinder opposite its current location (without servicing requests in between) and begins the next sweep. The C-LOOK policy is characterized by potentially lower variance of response times compared to LOOK; it offers high throughput (although lower than that of LOOK).

circular SCAN (C-SCAN) disk scheduling—Disk scheduling strategy that moves the arm in one direction, servicing requests on a shortest-seek basis. When the arm has completed its a sweep, it jumps (without servicing requests) to

the cylinder opposite its current location, then resumes its inward sweep processing requests. C-SCAN maintains high levels of throughput while further limiting variance of response times by avoiding the discrimination against the innermost and outermost cylinders.

coarse-grained strip (RAID)—Strip size that enables average files to be stored in a small number of strips. In this case, some requests can be serviced by only a portion of the disks in the array, so it is more likely that multiple requests can be serviced simultaneously. If requests are small, they are serviced by one disk at a time, which reduces their average transfer rates.

cylinder—Set of tracks that can be accessed by the read-write heads for a specific position of the disk arm.

data compression—Technique that decreases the size of a data record by replacing repetitive patterns with shorter bit strings. This can reduce seeks, and transmission times, but may require substantial processor time to compress the data for storage on the disk, and to decompress the data to make it available to applications.

data regeneration (RAID)—Reconstruction of lost data (due to disk errors or failures) in RAID systems.

data striping (RAID)—Technique in RAID systems that divides contiguous data into fixed-size strips that can be placed on different disks. This enables multiple disks to service requests for data.

defragmentation—Moving parts of files so that they are located in contiguous blocks on disk. This can reduce access times when reading from or writing to files sequentially.

disk arm—Moving-head disk component that moves read-write heads linearly, parallel to disk surfaces.

disk arm anticipation—Moving the disk arm to a location that will minimize the next seek. Disk arm anticipation can be useful in environments where process disk request patterns exhibit locality and when the load is light enough that there is sufficient time to move the disk arm between disk requests without degrading performance.

disk cache buffer—A region of main memory that the operating system reserves for disk data. In one context, the reserved memory acts as a cache, allowing processes quick access to data that would otherwise need to be fetched from disk. The reserved memory also acts as a buffer, allowing the operating system to delay writing the data to improve I/O performance by batching multiple writes into a small number of requests.

disk mirroring (RAID)—Data redundancy technique in RAID that maintains a copy of each disk's contents on a separate disk. This technique provides high reliability and

simplifies data regeneration but incurs substantial storage overhead, which increases cost.

disk reorganization—Technique that moves file data on disk to improve its access time. One such technique is defragmentation, which attempts to place sequential file data contiguously on disk. Another technique attempts to place frequently requested data on tracks that result in low average seek times.

disk scheduling—Technique that orders disk requests to maximize throughput and minimize response times and the variance of response times. Disk scheduling strategies improve performance by reducing seek times and rotational latencies.

elevator algorithm—See SCAN disk scheduling.

first-come-first-served (FCFS) disk scheduling—Disk scheduling strategy in which the earliest arriving request is serviced first. FCFS is a fair method of allocating service, but when the request rate (i.e., the load) becomes heavy, FCFS can result in long waiting times. FCFS exhibits a random seek pattern in which successive requests can cause time-consuming seeks from the innermost to the outermost cylinders.

fine-grained strip (RAID)—Strip size that causes average files to be stored in multiple stripes. Fine-grained strips can reduce each request's access time and increase transfer rates because multiple disks simultaneously retrieve portions of the requested data.

FSCAN disk scheduling—Disk scheduling strategy that uses SCAN to service only those requests waiting when a particular sweep begins (the "F" stands for "freezing" the request queue at a certain time). Requests arriving during a sweep are grouped together and ordered for optimum service during the return sweep.

fragmented disk—Disk that stores files in discontinuous blocks as the result of file creation and deletion. Such disks exhibit high seek times when reading files sequentially. Disk defragmentation can reduce or eliminate the problem.

Hamming error-correcting codes (Hamming ECCs)—Technique of generating parity bits that enables systems to detect and correct errors in data transmission.

hard disk drive—Magnetic, rotational secondary storage device that provides persistent storage for, and random access to, data.

hot spare disk (RAID)—Disk in a RAID system that is not used until a disk fails. Once the system regenerates the failed disk's data, the hot spare disk replaces the failed disk.

hot spot—Disk cylinder that contains frequently requested data. Some disk arm anticipation techniques move the disk head to hot spots to reduce average seek times.

load—See request rate.

level (RAID)—A particular organization of a RAID system, such as level 1 (mirroring) or level 2 (Hamming ECC parity). See also RAID level 0, RAID level 1, RAID level 2, RAID level 3, RAID level 4 and RAID level 5.

LOOK disk scheduling—Variation of the SCAN disk scheduling strategy that "looks" ahead to the end of the current sweep to determine the next request to service. If there are no more requests in the current direction, LOOK changes the preferred direction and begins the next sweep, stopping when passing a cylinder that corresponds to a request in the queue. This strategy eliminates unnecessary seek operations experienced by other variations of SCAN by preventing the read-write head from moving to the innermost or outermost cylinder unless it needs to service a request at those cylinders.

magnetic tape storage—Rewritable magnetic storage medium that accesses data sequentially. Its sequential nature makes it unsuitable for direct access applications.

mean response time (disk scheduling)—Average time a system spends waiting for a disk request to be serviced.

mean-time-to-failure (MTTF) (RAID)—Average time before a single-disk failure.

mirroring (RAID)—See disk mirroring.

moving-arm assembly—See disk arm.

nonuniform request distribution—Set of disk requests that are not uniformly distributed across disk surfaces. This occurs because processes exhibit spatial locality, leading to localized request patterns.

N-Step SCAN disk scheduling—Disk scheduling strategy that services the first n requests in the queue using the SCAN strategy. When the sweep is complete, the next n requests are serviced. Arriving requests are placed at the end of the request queue. N-Step SCAN offer good performance due to high throughput, low mean response times and a lower variance of response times than SSTF and SCAN.

online spare—See hot spare disk.

online transaction processing (OLTP)—Type of system that typically receives many disk requests to randomly distributed locations containing small amounts of data (e.g., databases and Web servers). Such systems significantly improve performance using disk scheduling algorithms.

partition—Area of a disk whose boundaries cannot be crossed by file data. Partitions can reduce disk fragmentation.

parity—Technique that detects an even number of errors in data transmission. Parity information is generated by determining whether the data contains an even (or odd) number of 1's (or 0's). This parity information is generated after transmission and compared to the value generated before transmission. Error-correction codes (ECCs), such as Hamming or XOR, use the parity of a string of bits to detect and correct errors. Parity enables RAID systems to provide fault tolerance with lower storage overhead than mirrored systems.

parity logging (RAID)—Technique that increases write performance in RAID systems using parity by postponing writes to the parity disk while the system is busy. Because parity logging stores information in volatile memory, parity information could be lost if the system loses power.

platter—Magnetic disk medium that stores bits on its surfaces.

positioning time—Access time plus latency. Positioning time is used by the SPTF strategy to order requests.

preferred direction—Direction in which the disk head is moving in SCAN-based scheduling algorithms.

RAID (Redundant Array of Independent Disks)—Family of techniques that use an array of disks to improve disk transfer rates while providing fault tolerance.

RAID controller—Special-purpose hardware that efficiently performs operations such as dividing files into strips, forming files from strips, determining the locations of strips in the array and implementing the array's fault-tolerance mechanism.

RAID level 0—RAID system that uses a striped disk array with no redundancy. Level 0 arrays are not fault tolerant: if one disk fails, all the data in the array that depend on the failed disk are lost. Depending on the array's strip size, all data stored in the array could become unusable with the loss of a single disk. Although RAID 0 is not fault tolerant, it is simple to implement, yields high transfer rates and does not incur any storage overhead. Level 0 arrays are implemented in systems where performance is more important than fault tolerance, such as supercomputers.

RAID level 1—RAID system that employs disk mirroring (also called shadowing) to provide redundancy, so that each disk in the array is duplicated. Stripes are not implemented in level 1, reducing both hardware complexity and system performance. While this results in the highest degree of fault tolerance of any RAID level, only half of the array's capacity can be used to store unique data, which increases cost. Level 1 arrays are implemented in systems where high availability is more important than cost, such as database systems.

RAID level 2—RAID system that is striped at the bit level (i.e., each strip stores one bit). Level 2 arrays are designed to reduce the storage overhead incurred by implementing fault tolerance using mirroring. Instead of maintaining redundant copies of each data item, RAID level 2 uses a version of Hamming error-correcting codes (Hamming ECCs) to store parity information that allows the system to detect up to two errors, correct up to one error and determine the location of the error in a stripe. The size of Hamming ECC codes, and thus the number of parity disks, increases according to the logarithm (base 2) of the number of data disks. Thus, level 2 arrays containing a large number of disks incur significantly less storage overhead than level 1 arrays.

RAID level 3—RAID system that stripes data at the bit or byte level. RAID 3 uses XOR (exclusive or) error-correcting codes (XOR ECCs), which use the logical XOR operation to generate parity information. XOR ECC uses only one disk to hold parity information, regardless of the size of the array. The system can use the parity bits to recover from any single disk failure. Due to parity checking, RAID level 3 reads and writes require access to the entire array. Similar to RAID level 2, this yields high transfer rates when reading and writing large files, but only one request can be serviced at a time.

RAID level 4—RAID system that is striped using fixed-size blocks (typically larger than a byte) and uses XOR ECC to generate parity data that is stored on a single parity disk. Because level 4 arrays enable coarse-grained striping, the system can potentially service multiple read requests simultaneously, if parity is not determined for each read. When servicing a write request, however, the system must update parity information to ensure that no data is lost in the event of a disk failure. This means that write requests must be performed one at a time, creating a write bottleneck.

RAID level 5—RAID system that is striped at the block level and uses XOR ECC for parity, but distributes parity blocks throughout the array of disks. Because parity blocks are distributed across many disks, multiple parity strips can be accessed simultaneously, removing the write bottleneck for many requests. Although RAID level 5 increases performance relative to RAID levels 2-4, it is complex to implement and more costly. Level 5 arrays are considered general-purpose arrays and are often found in file and application servers, enterprise resource planning (ERP) and other business systems.

RAMAC (Random Access Method of Accounting and Control)—First commercial hard drive produced by IBM.

random seek pattern—Series of requests to cylinders randomly distributed across disk surfaces. FCFS causes random seek patterns that result in high response times and low throughput.

read-modify-write cycle (RAID)—Operation that reads a stripe, modifies its contents and parity, then writes the stripe to the array. It is performed for each write request in RAID systems that use parity. Some systems reduce the cost of read-modify-write cycles by caching strips or by updating parity information only periodically.

read-write head—Moving-head disk component that hovers over a disk surface, reading and writing bits as the disk moves.

redundancy—Technique that maintains multiple identical resources to enable recovery upon failure.

Redundant Array of Independent Disks (RAID)—See RAID.

reliability—Measure of fault tolerance. The more reliable a resource, the less likely it is to fail.

request rate—Measure of request frequency. The higher the request rate, the greater the system load. Systems experiencing a high disk request rate tend to benefit from disk scheduling.

rotational latency—Time required for a disk to rotate a requested data item from its current position to a position adjacent to the read-write head.

rotational optimization—Disk scheduling technique that reduces access times by next servicing the request to the nearest sector in the read-write head's current cylinder.

SCAN disk scheduling—Disk scheduling strategy that reduces unfairness and variance of response times as compared to SSTF by servicing the request that requires the shortest seek distance in a preferred direction. SCAN behaves much like SSTF in terms of high throughput and good mean response times. However, because SCAN ensures that all requests in a given direction will be serviced before the requests in the opposite direction, it offers a lower variance of response times than SSTF. Also called the elevator algorithm.

sector—Smallest portion of a track that can be accessed by an I/O request.

sector queuing—See rotational optimization.

seek operation—Operation that moves the disk head to a different cylinder.

seek time—Time it takes for the read/write head to move from its current cylinder to the cylinder containing the requested data record.

574 Disk Performance Optimization

seek optimization—Disk scheduling technique that reduces seek times by generally servicing requests to the cylinder near the read-write head.

shadowing (RAID)—See mirroring.

shortest-access-time-first (SATF) disk scheduling—Disk scheduling strategy that next services the request that requires the shortest access time (i.e., positioning time plus transmission time). SATF exhibits higher throughput than SPTF, but large requests can be indefinitely postponed by a series of smaller requests, and requests to the innermost or outermost cylinders can be indefinitely postponed by requests to intermediate cylinders.

shortest-latency-time-first (SLTF) disk scheduling—Disk scheduling strategy that examines all of the waiting requests and services the one with the shortest rotational delay first. This strategy has been shown to be close to the theoretical optimum and is relatively easy to implement.

shortest-positioning-time-first (SPTF) disk scheduling—Disk scheduling strategy that next services the request that requires the shortest positioning time. SPTF results in high throughput and a low mean response time, similar to SSTF, and can also indefinitely postpone requests to the innermost and outermost cylinders.

shortest-seek-time-first (SSTF) disk scheduling—Disk scheduling strategy that next services the request that is closest to the read-write head's current cylinder (and thus incurs the shortest seek time), even if that is not the first one in the queue. By reducing average seek times, SSTF achieves higher throughput rates than FCFS, and mean response times tend to be lower for moderate loads. One significant drawback is that higher variances occur on response times because of the discrimination against the outermost and innermost tracks; in the extreme, starvation of requests far from the read-write heads could occur.

spindle—Moving-head disk component that spins platters at high speeds.

strip (RAID)—Smallest unit of data operated on by a RAID system. The set of strips at the same location on each disk is called a stripe.

stripe (RAID)—Set of strips at the same location on each disk in a RAID system. Striping enables RAID systems to access files using multiple disks at once, which improves transfer times.

track—Circular region of data on a platter. Sequential file data is typically placed contiguously on one track to improve access time by reducing seeking activity.

transmission time—Time required for a data record to pass by the read-write head.

UNIVAC 1 (UNIVersal Automatic Computer)—First computer to introduce a magnetic storage tape.

update image (RAID)—Method to reduce parity computation time by storing the difference between new and old parities in memory instead of performing a read-modify-write cycle.

write-back caching—Technique that writes buffered data to disk periodically, enabling the operating system to batch multiple I/Os that are serviced using a single request, which can improve system performance.

write-through caching—Technique that writes data both to the disk cache buffer and to disk each time cached data is modified. This technique prevents the system from batching requests, but reduces the possibility of inconsistent data in the event of a system crash.

XOR (exclusive-or) operation—Operation on two bits that returns 1 if the two bits are not the same, 0 otherwise. RAID levels 3-5 use the XOR operation to generate parity bits.

Exercises

12.1 What are the essential goals of disk scheduling? Why is each important?

12.2 Suppose that on a particular model of disk drive, average seek times approximately equal average latency times. How might this affect a designer's choice of disk scheduling strategies?

12.3 What makes a given disk scheduling discipline fair? Just how important is fairness compared to other goals of disk scheduling disciplines?

12.4 Disk scheduling disciplines that are not fair generally have a larger variance of response times than FCFS. Why is this true?

12.5 Under light loading conditions, virtually all the disk scheduling disciplines discussed in this chapter degenerate to which scheme? Why?

12.6 One criterion that influences the desirability of implementing disk scheduling is the disk scheduling mechanism's runtime overhead. What factors contribute to this overhead? Suppose t is the average seek time on a system with FCFS disk

scheduling. Suppose s is the approximate time it would take to schedule an average disk request if another form of disk scheduling were in use. Assume all other factors are favorable to incorporating disk scheduling on this system. For each of the following cases, comment on the potential effectiveness of incorporating disk scheduling on the system.

- a. $5 = 0.01t$
- b. $s = 0.1t$
- c. $s = t$
- d. $s = 10t$

12.7 Latency optimization usually has little effect on system performance except under extremely heavy loads. Why?

12.8 On interactive systems it is essential to ensure reasonable response times to users. Reducing the variance of response times is still an important goal, but it is not sufficient to prevent an occasional user from suffering indefinite postponement. What additional mechanism would you incorporate into a disk scheduling discipline on an interactive system to help ensure reasonable response times and to avoid the possibility of indefinite postponement?

12.9 Argue why some people refuse to consider FCFS to be a disk scheduling discipline. Is there a scheme for servicing disk requests that we would be even less inclined to call disk scheduling than FCFS?

12.10 In what sense is LOOK fairer than SSTF? In what sense is C-LOOK fairer than LOOK?

12.11 Give a statistical argument for why FCFS offers a small variance of response times.

12.12 Argue why SSTF tends to favor requests to midrange tracks at the expense of the requests to the innermost and outermost tracks.

12.13 Chapter 7, Deadlock and Indefinite Postponement, suggests a scheme for avoiding indefinite postponement. Suggest an appropriate modification to the SSTF scheme to create a "nonstarving SSTF." Compare this new version with regular SSTF with regard to throughput, mean response times, and variance of response times.

12.14 It is possible that while a disk request for a particular cylinder is being serviced, another request for that cylinder will arrive. Some disk scheduling disciplines would service this new request immediately after processing the current request. Other disciplines preclude the servicing of the new request until the return sweep of the disk arm. What dangerous situation might occur in a disk scheduling discipline that allowed

immediate servicing of a new request for the same cylinder as the previous request?

12.15 Why does SCAN have a lower variance of response times than SSTF?

12.16 Compare the throughput of FSCAN with that of SCAN.

12.17 Compare the throughput of C-SCAN with that of SCAN.

12.18 How does a latency optimization scheme operate?

12.19 A disk storage bottleneck may not always be removed by adding more disk drives. Why?

12.20 How does the level of multiprogramming affect the need for disk scheduling?

12.21 Suppose controller saturation is indicated. Might disk scheduling be useful? What other actions should be taken?

12.22 Why is it desirable to assume uniform request distributions when considering disk scheduling disciplines? In what types of systems might you expect to observe relatively uniform request distributions? Give several examples of systems that tend to have nonuniform request distributions.

12.23 Would disk scheduling be useful for a single-user, dedicated disk in a sequential file processing application? Why?

12.24 The VSCAN scheduling strategy combines SSTF and SCAN according to a variable R .⁹¹ VSCAN determines the next request to service using SSTF, but each request to a cylinder that requires the read/write head to change direction is weighted by a factor R . The "distances" to requests in the opposite direction are computed as follows. If C is the number of cylinders on the disk and S is the number of cylinders between the current location of the head and the next request, then VSCAN uses the distance, $D = S + R \times C$. Thus, VSCAN uses R to add an extra cost to the requests in the unpreferred direction.

12.25 If $R = 0$, to which disk scheduling policy does VSCAN degenerate?

12.26 If $R = 1$, to which disk scheduling policy does VSCAN degenerate?

12.27 Discuss the benefit of choosing a value, R , somewhere between 0 and 1.

12.28 In what circumstances might disk scheduling actually result in poorer performance than FCFS?

12.29 Compare the essential goals of disk scheduling with those of processor scheduling. What are the similarities? What are the differences?

12.30 What factors contribute to the need for seeking?

576 Disk Performance Optimization

12.31 A sequential file "properly" stored on an empty disk can be retrieved with minimal seeking. But disks tend to become heavily fragmented as files are added and deleted. Thus, successive references to "adjacent" records in a disk file can result in substantial seeking throughout the disk. Discuss the notion of file reorganization as a scheme for minimizing seeks. Under what circumstances should a file be reorganized? Should every file on the disk be reorganized or should this operation be limited to certain files only?

12.32 A designer has proposed the use of multiple disk arms to greatly improve the response times of a disk subsystem. Discuss the advantages and disadvantages of such a scheme.

12.33 We mentioned that the LOOK algorithm is like the algorithm followed by elevators in buildings. In what ways is this an appropriate analogy?

12.34 Suggest other techniques besides disk scheduling and file reorganization for minimizing seeks.

12.35 The SCAN strategy often exhibits significantly higher throughput than the C-SCAN algorithm. However, system designers sometimes choose C-SCAN over SCAN. Why might this decision be justified?

12.36 In Section 12.10.6, we said that a RAID system can reduce the number of I/O operations when calculating parity by using the formula $A_p' = (A_d \text{ XOR } A_d') \text{ XOR } A_p$. Prove that this relation holds for all strips A_d , A_d' and A_p .

12.37 For each of the following applications, indicate whether the system should use RAID level 0, RAID level 1, or RAID level 5 and explain why.

12.38 Storing account transactions for a financial institution

12.39 Storing high-resolution telescope images (that have been transferred from a tape drive) before processing on a supercomputer

12.40 Running a personal Web server

Suggested Projects

12.41 Research disk scheduling in Linux. What scheduling strategies does Linux use? How are these strategies implemented? For an introduction to Linux disk scheduling, see Section 20.8.3, Block Device I/O.

12.42 Prepare a research paper discussing the techniques that hard disk manufacturers employ to improve performance. See the links in the Web Resources section.

12.43 Prepare a report briefly describing five RAID levels other than the 6 covered in detail in this chapter.

Suggested Simulations

12.44 Develop monitors (see Chapter 6, Concurrent Programming) to implement each of the following disk scheduling disciplines.

- a. FCFS
- b. SSTF
- c. SCAN
- d. N-Step SCAN
- e. C-SCAN

12.45 In this chapter, we have investigated several cylinder-oriented disk scheduling strategies including FCFS, SSTF, SCAN, TV-Step SCAN, and C-SCAN, F-SCAN, LOOK and C-LOOK. Write a simulation program to compare the performance of these scheduling methods under light, medium, and heavy loads. Define a light load as one in which there is usually either 0 or 1 requests waiting for the entire disk. Define a medium load as one in which each cylinder has either 0 or 1 request waiting. Define a heavy load as one in which many cylinders are likely to have many requests pending. Use random

number generation to simulate the arrival time of each request as well as the actual cylinder for each request.

12.46 The availability of moving-head disk storage has contributed greatly to the success of modern computing systems. Disks offer relatively fast and direct access to enormous amounts of information. Industry forecasters see a solid future for disk-based secondary storage systems, and researchers continue developing new and improved disk scheduling disciplines. Propose several new disk scheduling schemes and compare them with those presented in this chapter.

Develop a program that simulates a disk that contains two platters (i.e., four surfaces), four read/write heads, 25 cylinders and 20 sectors per track. This disk should store the current position of the read/write head and provide functions such as read, write and seek. These functions should return the number of milliseconds required to service each request (track-to-track seek times of 2 ms, average seek times of 10 ms and average latencies of 5 ms are reasonable). Then create a simulation program that generates a uniform distribution of requests specifying a location by platter, cylinder and sector and a size by the number of requested sectors. These requests

should be sent to your disk scheduler, which then orders them and sends them to the disk for service by calling its read, write and seek functions. Use the values returned by the disk to determine each scheduling algorithm's throughput, response time and variance of response times.

Recommended Reading

Because disks are a bottleneck in most systems, disk performance optimization continues to be an active area of research and analysis in the literature. Denning discussed how disk scheduling algorithms can improve performance.⁹² As hard disks became the dominant form of secondary storage, research into improving their performance continued.^{93, 94, 95, 96, 97, 98, 99} Researchers regularly perform comparative analyses of disk scheduling algorithms, often using sophisticated simulations of hard disk activity.³⁶⁻³⁹ Patterson et al. introduced the notion of

12.47 Extend the simulation in the previous exercise to incorporate rotational latency. For each strategy, did performance increase or decrease as a result?

RAID in their seminal paper,¹⁰⁰ subsequent research has further explored the trade-offs between various RAID levels.¹⁰¹⁻¹⁰² As the gap between main memory and hard disk speeds grows at an increasing pace, microelectromechanical storage (MEMS) devices, which access a rectangular media using chips containing thousands of read/write heads, show promise as a technology to replace the hard disk as high-capacity, low-cost storage.^{103,104} The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. Kozierok, C, "Life Without Hard Disk Drives," *PCGuide*, April 17, 2001, <www.pcguid.com/ref/hdd/histWithout-c.html>.
2. Khurshudov, A., *The Essential Guide to Computer Data Storage*, Upper Saddle River, NJ: Prentice Hall PTR., 2001, p. 6.
3. "RAMAC," *Whatis.com*, April 30, 2001, <searchstorage.techtarget.com/sDefinition/0,,sid5_gci548619,00.html>.
4. Khurshudov, A., *The Essential Guide to Computer Data Storage*, Upper Saddle River, NJ: Prentice Hall PTR, 2001, p. 90.
5. "Cost of Hard Drive Space," September 20, 2002, <www.littletechshop.com/ns1625/winchest.html>.
6. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1998, p. 109.
7. Gotlieb, C, and G. MacEwen, "Performance of Movable-Head Disk Storage Devices," *Journal of the ACM*, Vol. 20, No. 4, October 1973, pp. 604-623.
8. Smith, A., "On the Effectiveness of Buffered and Multiple Arm Disks," *Proceedings of the Fifth Symposium on Computer Architecture*, 1978, pp. 109-112.
9. Pechura, M., and J. Schoeffler, "Estimated File Access Time of Floppy Disks," *Communications of the ACM*, Vol. 26, No. 10, October 1983, pp. 754-763.
10. Kozierok, C, "Spindle Speed," *PCGuide*, April 17, 2001, <www.pcguid.com/ref/hdd/op/spinSpeed-c.html>.
11. Walters, S., "Estimating Magnetic Disc Seeks," *Computer Journal*, Vol. 18, No. 1, 1973, pp. 412-416.
12. Kollias, J., "An Estimate of Seek Time for Batched Searching of Random or Indexed Sequential Structured Files," *Computer Journal*, Vol. 21, No. 2, 1978, pp. 21-26.
13. Kozierok, C, "Hard Disk Tracks, Cylinders and Sectors," modified April 17, 2001, <www.pcguid.com/ref/hdd/geom/tracks.htm>.
14. "DiamondMax Plus 9," Maxtor Corporation, May 14, 2003, <www.maxtor.com/en/documentation/data_sheets/diamondmax_plus_9_data_sheet.pdf>.
15. "WD Caviar Hard Drive 250GB 7200 RPM," October 28, 2003, <www.westerndigital.com/en/products/Products.asp?ProductId=41>.
16. "MK8025GAS," Toshiba America, Inc., October 28, 2003, <sdd.toshiba.com/main.aspx?Path=/818200000007000000010000659800001516/81820000011d000000010000659c000003fd/8182000001c8000000010000659c00000599/8182000001e5000000010000659c000005cb/8182000006db000000010000659c00001559>.
17. "WD Raptor Enterprise Serial ATA Hard Drive 36.7 GB 10,000 RPM," October 28, 2003, <www.westerndigital.com/en/products/WD360CD.asp>.
18. "Cheetah 15K.3," *Seagate Technology*, March 27, 2003, <www.Seagate.com/docs/pdf/datasheet/disc/ds_cheetah15k3.pdf>.
19. Wilhelm, N., "An Anomaly in Disk Scheduling: A Comparison of FCFS and SSTF Seek Scheduling Using an Empirical Model for Disk Accesses," *Communications of the ACM*, Vol. 19, No. 1, January 1976, pp. 13-17.
20. Wong, C, "Minimizing Expected Head Movement in One-Dimensional and Two-Dimensional Mass Storage Systems," *ACM Computing Survey's*, Vol. 12, No. 2, 1980, pp. 167-178.
21. Frank, H, "Analysis and Optimization of Disk Storage Devices for Time-Sharing Systems," *Journal of the ACM*, Vol. 16, No. 4, October 1969, pp. 602-620.

22. Teorey, T., "Properties of Disk Scheduling Policies in Multiprogrammed Computer Systems," *Proceedings of AFIPS FJCC*, Vol. 41, 1972, pp. 1-11.
23. Wilhelm, N., "An Anomaly in Disk Scheduling: A Comparison of FCFS and SSTF Seek Scheduling Using an Empirical Model for Disk Access," *Communications of the ACM*, Vol. 19, No. 1, January 1976, pp. 13-17.
24. Hofri, M., "Disk Scheduling: FCFS vs. SSTF Revisited," *Communications of the ACM*, Vol. 23, No. 11, November 1980, pp. 645-653.
25. Denning, P., "Effects of Scheduling on File Memory Operations," *Proceedings of AFIPS, SJCC*, Vol. 30, 1967, pp. 9-21.
26. Teorey, T., "Properties of Disk Scheduling Policies in Multiprogrammed Computer Systems," *Proceedings of AFIPS FJCC*, Vol. 41, 1972, pp. 1-11.
27. Wilhelm, N., "An Anomaly in Disk Scheduling: A Comparison of FCFS and SSTF Seek Scheduling Using an Empirical Model for Disk Access," *Communications of the ACM*, Vol. 19, No. 1, January 1976, pp. 13-17.
28. Hofri, M., "Disk Scheduling: FCFS vs. SSTF Revisited," *Communications of the ACM*, Vol. 23, No. 11, November 1980, pp. 645-653.
29. Denning, P., "Effects of Scheduling on File Memory Operations," *Proceedings of AFIPS, SJCC*, Vol. 30, 1967, pp. 9-21.
30. Teorey, T., "Properties of Disk Scheduling Policies in Multiprogrammed Computer Systems," *Proceedings of AFIPS FJCC*, Vol. 41, 1972, pp. 1-11.
31. Gotlieb, C. and G. MacEwen, "Performance of Movable-Head Disk Storage Devices," *Journal of the ACM*, Vol. 20, No. 4, October 1973, pp. 604-623.
32. Teorey, T. and T. Pinkerton, "A Comparative Analysis of Disk Scheduling Policies," *Communications of the ACM*, Vol. 15, No. 3, March 1972, pp. 177-184.
33. Thomasian, A., and C. Liu, "Special Issue on the PAPA 2002 Workshop: Disk Scheduling Policies with Lookahead," *ACM SIGMETRICS Performance Evaluation Review*, Vol. 30, No. 2, September 2002, pp. 36.
34. Teorey, T., "Properties of Disk Scheduling Policies in Multiprogrammed Computer Systems," *Proceedings of AFIPS FJCC*, Vol. 41, 1972, pp. 1-11.
35. Teorey, T. and T. Pinkerton, "A Comparative Analysis of Disk Scheduling Policies," *Communications of the ACM*, Vol. 15, No. 3, March 1972, pp. 177-184.
36. Worthington, B.; G. Ganger; and Y. Patt, "Scheduling Algorithms for Modern Disk Drives," *Proceedings of the 1994 ACM SIGMETRICS Conference*, May 1994, p. 243.
37. "Hard Disk Specifications," viewed October 2, 2003, <www.storage-review.com/guide2000/ref/hdd/perf/perf/spec/index.html>.
38. Stone, H., and S. Fuller, "On the Near Optimality of the Shortest-Latency-Time-First Drum Scheduling Discipline," *Communications of the ACM*, Vol. 16, No. 6, June 1973, pp. 352-353.
39. Thomasian, A., and C. Liu, "Special Issue on the PAPA 2002 Workshop: Disk Scheduling Policies with Look-Ahead," *ACM SIGMETRICS Performance Evaluation Review*, Vol. 30, No. 1, September 2002, p. 33.
40. Thomasian, A., and C. Liu, "Special Issue on the PAPA 2002 Workshop: Disk Scheduling Policies with Look-Ahead," *ACM SIGMETRICS Performance Evaluation Review*, Vol. 30, No. 2, September 2002, p. 33.
41. Kozierok, C., "Logical Geometry," *PCGuide*, April 17, 2001, <www.pcguides.com/refhdd/perf/perf/extp/pcCaching-c.html>.
42. Khurshudov, A., *The Essential Guide to Computer Data Storage* (Upper Saddle River, NJ: Prentice Hall PTR, 2001), pp. 106-107.
43. "Dell - Learn More - Hard Drives," 2003, <www.dell.com/us/en/dhs/learnmore/learnmore_hard_drives_desktop_popup_dimen.htm>.
44. Chaney, R., and B. Johnson, "Maximizing Hard-Disk Performance: How Cache Memory Can Dramatically Affect Transfer Rate," *Byte*, May 1984, pp. 307-334.
45. Thomasian, A., and C. Liu, "Special Issue on the PAPA 2002 Workshop: Disk Scheduling Policies with Look-Ahead," *ACM SIGMETRICS Performance Evaluation Review*, Vol. 30, No. 2, September 2002, p. 38.
46. Wilhelm, N., "An Anomaly in Disk Scheduling: A Comparison of FCFS and SSTF Seek Scheduling Using an Empirical Model for Disk Accesses," *Communications of the ACM*, Vol. 19, No. 1, January 1976, pp. 13-17.
47. Lynch, W., "Do Disk Arms Move?" *Performance Evaluation Review, ACM Sigmetrics Newsletter*, Vol. 1, December 1972, pp. 3-16.
48. Kozierok, C., "Cache Write Policies and the Dirty Bit," *PCGuide*, April 17, 2001, <www.pcguides.com/ref/mbsys/cache/funcWrite-c.html>.
49. Thomasian, A., and C. Liu, "Special Issue on the PAPA 2002 Workshop: Disk Scheduling Policies with Look-Ahead," *ACM SIGMETRICS Performance Evaluation Review*, Vol. 30, No. 2, September 2002, p. 32.
50. Kozierok, C., "Operating System and Controller Disk Caching," *PCGuide*, April 17, 2001, <www.pcguides.com/refhdd/perf/perf/extp/pcCaching-c.html>.
51. Thomasian, A., and C. Liu, "Special Issue on the PAPA 2002 Workshop: Disk Scheduling Policies with Look-Ahead," *ACM SIGMETRICS Performance Evaluation Review*, Vol. 30, No. 2, September 2002, p. 31.
52. "Disk Optimization Can Save Time and Resources in a Windows NT/2000 Environment," *Raxco Software*, <www.raxco.dk/raxco/perfectdisk2000/download/Optimization_Can_Save_Time.pdf>.
53. King, R., "Disk Arm Movement in Anticipation of Future Requests," *ACM Transactions in Computer Systems*, Vol. 8, No. 3, August 1990, p. 215.
54. King, R., "Disk Arm Movement in Anticipation of Future Requests," *ACM Transactions in Computer Systems*, Vol. 8, No. 3, August 1990, p. 214.

- 55- King, R., "Disk Arm Movement in Anticipation of Future Requests," *ACM Transactions in Computer Systems*, Vol. 8, No. 3, August 1990, pp. 220,226
56. "RAID Overview," <www.amsstorage.com/html/raid_overview.html>.
57. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 109.
58. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, pp. 109-110.
59. Chen, P.; E. Lee; G. Gibson; R. Katz; and D. Patterson, "RAID: High-Performance, Reliable Secondary Storage," *ACM Computing Surveys*, Vol. 26, No. 2, June 1994, pp. 151-152.
60. Chen, P.; E. Lee; G. Gibson; R. Katz; and D. Patterson, "RAID: High-Performance, Reliable Secondary Storage," *ACM Computing Surveys*, Vol. 26, No. 2, June 1994, p. 147.
61. Chen, P.; E. Lee; G. Gibson; R. Katz; and D. Patterson, "RAID: High-Performance, Reliable Secondary Storage," *ACM Computing Surveys*, Vol. 26, No. 2, June 1994, p. 152.
- fc. "RAID 0: Striped Disk Array without Fault Tolerance," <www.raid.com/04_01_00.html>.
63. "RAID 0: Striped Disk Array without Fault Tolerance," <www.raid.com/04_01_00.html>.
64. Chen, P.; E. Lee; G. Gibson; R. Katz; and D. Patterson, "RAID: High-Performance, Reliable Secondary Storage," *ACM Computing Surveys*, Vol. 26, No. 2, June 1994, p. 152.
65. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 112.
66. "RAID Overview," <www.masstorage.com.com/html/raid_overview.html>.
67. "RAID 1: Mirroring and Duplexing," <www.raid.com/04_01_01.html>.
68. "RAID 2: Hamming Code ECC," <www.raid.com/04_01_02.html>.
69. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 112.
70. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 112.
71. "RAID Overview," <www.amsstorage.com.com/html/raid_overview.html>.
72. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 112.
73. Chen, P.; E. Lee; G. Gibson; R. Katz; and D. Patterson, "RAID: High-Performance, Reliable Secondary Storage," *ACM Computing Surveys*, Vol. 26, No. 2, June 1994, p. 156.
74. "RAID 3: Parallel Transfer with Parity," <www.raid.com/04_01_03.html>.
- 75- Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOA* June 1988, p. 113.
76. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 113.
77. "RAID Overview," <www.amsstorage.com/html/raid_overview.html>.
78. "RAID 5: Independent Data Disks with Distributed Parity Blocks," <www.rand.com/04_01_05.html>.
79. "RAID Level 4," *PCGuide.com*, <www.pcguid.com/ref/hdd/perf/rand/levels/singleLevel4-c.html>.
80. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 114.
81. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 114.
82. Savage, S., and J. Wilkes, "AFRAID-A Frequently Redundant Array of Independent Disks," *Proceedings of the 1996 Usenix Conference*, January 1996, p. 27.
83. Chen, P.; E. Lee; G. Gibson; R. Katz; and D. Patterson, "RAID: High-Performance, Reliable Secondary Storage," *ACM Computing Surveys*, Vol. 26, No. 2, June 1994, pp. 166-168.
84. Savage, S., and J. Wilkes, "AFRAID-A Frequently Redundant Array of Independent Disks," *Proceedings of the 1996 Usenix Conference*, January 1996, pp. 27,37.
85. "RAID 5: Independent Data Disks with Distributed Parity Blocks," <www.raid.com/04_01_05.html>.
86. "RAID Level 5," *PCGuide.com*, <www.pcguid.com/ref/hdd/perf/raid/levels/singleLevel5-c.html>.
87. "Multiple (Nested) RAID Levels," *PCGuide.com*, <www.pcguid.com/ref/hdd/perf/raid/mult.htm>.
88. "RAID 6: Independent Data Disks with Two Independent Parity Schemes," <www.raid.com/04_01_06.html>.
89. "Multiple (Nested) RAID Levels," *PCGuide.com*, <www.pcguid.com/ref/hdd/perf/raid/mult.htm>.
90. "RAID Level 7," *PCGuide.com*, <www.pcguid.com/ref/hdd/perf/raid/levels/singleLevel7-c.html>.
91. Geist, R., and S. Daniel, "A Continuum of Disk Scheduling Algorithms," *ACM Transactions in Computer Systems*, Vol. 5, February 1,1987, p. 78.
92. Denning, P., "Effects of Scheduling on File Memory Operations," *Proceedings of AFIPS, SJCC*, Vol. 30,1967, pp. 9-21.
93. Teorey, T., "Properties of Disk Scheduling Policies in Multiprogrammed Computer Systems," *Proceedings of AFIPS FJCC*, Vol. 41,1972, pp. 1-11.
94. Wilhelm, N., "An Anomaly in Disk Scheduling: A Comparison of FCFS and SSTF Seek Scheduling Using an Empirical Model for Disk Access," *Communications of the ACM*, Vol. 19, No. 1, January 1976, pp. 13-17.

580 Disk Performance Optimization

- 95- Hofri, M., "Disk Scheduling: FCFS vs. SSTF Revisited," *Communications of the ACM*, Vol. 23, No. 11, November 1980, pp. 645-653.
96. Geist, R., and S. Daniel, "A Continuum of Disk Scheduling Algorithms," *ACM Transactions in Computer Systems*, Vol. 5, February 1, 1987, p. 78.
97. King, R., "Disk Arm Movement in Anticipation of Future Requests," *ACM Transactions in Computer Systems*, Vol. 8, No. 3, August 1990, p. 215.
98. Thomasian, A., and C. Liu, "Special Issue on the PAPA 2002 Workshop: Disk Scheduling Policies with Look-Ahead," *ACM SIGMETRICS Performance Evaluation Review*, Vol. 30, No. 2, September 2002, p. 33.
99. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 109.
100. Patterson, D.; G. Gibson; and R. Katz, "A Case for Redundant Arrays of Inexpensive Disks," *Proceedings of the ACM SIGMOD*, June 1988, p. 109.
101. Chen, P.; E. Lee; G. Gibson; R. Katz; and D. Patterson, "RAID High-Performance, Reliable Secondary Storage," *ACM Computing Surveys*, Vol. 26, No. 2, June 1994, pp. 151-152.
102. Savage, S., and J. Wilkes, "AFRAID—A Frequently Redundant Array of Independent Disks," *Proceedings of the 1996 Usenix Conference*, January 1996, pp. 27,37.
103. Griffin, J., L.; S. W. Schlosser; G. R. Ganger; and D. F. Nagle. "Operating System Management of MEMS-based Storage Devices," *Proceedings of the Fourth Symposium on Operating Systems Design and Implementation (OSDI)*, 2000, pp. 227-242.
104. Uysal, M.; A. Merchant; and G. A. Alvarez, "Using MEMS-based Storage in Disk Arrays," *Proceedings of the Second USENIX Conference on File and Storage Technologies*, March, 2003.

'Tis in my memory lock'd, And you yourself shall keep the key of it.

— William Shakespeare—

E pluribus unus. (One composed of many.)

—Virgil—

I can only assume that a "Do Not File" document is filled in a "Do Not File" file.

—Senator Frank Church, Senate Intelligence Subcommittee Hearing, 1975—

A form of government that is not the result of a long sequence of shared experiences, efforts, and endeavors can never take root.

—Napoleon Bonaparte—

Chapter 13

File and Database Systems

Objectives

After reading this chapter, you should understand:

- *the need for file systems.*
- *files, directories and the operations that can be performed on them.*
- *organizing and managing a storage device's data and free space.*
- *controlling access to data in a file system.*
- *backup, recovery and file system integrity mechanisms.*
- *database systems and models.*

Chapter Outling

13.1

Introduction

13.2

Data Hierarchy

13.3

Files

13.4

File Systems

Operating Systems Thinking:

Encryption and Decryption

13.4.1 Directories

13.4.2 Metadata

13.4.3 Mounting

13.5

File Organization

13.6

File Allocation

13.6.1 Contiguous File Allocation

13.6.2 Linked-List Noncontiguous File Allocation

13.6.3 Tabular Noncontiguous File Allocation

Mini Case Study: MS-DOS

13.6.4 Indexed Noncontiguous File Allocation

13.7

Free Space Management

13.8

File Access Control

Operating Systems Thinking: Security

13.8.1 Access Control Matrix

13.8.2 Access Control by User Classes

13.9

Data Access Techniques

13.10

Data Integrity Protection

13.10.1 Backup and Recovery

Operating Systems Thinking: Backup and Recovery

13.10.2 Data Integrity and Log-Structured File Systems

Operating Systems Thinking: Murphy's Law and Robust Systems

13.11

File Servers and Distributed Systems

13.12 ***Database Systems***

13.12.1 Advantages of Database Systems

13.12.2 Data Access

13.12.3 Relational Database Model

*13.12.4 Operating Systems and Database
Systems*

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

13.1 Introduction

Most computer users are familiar with the concept of file as a named collection of data that is manipulated as a unit. Files typically reside on secondary storage devices such as disks, CDs or tapes, though they can exist exclusively in volatile main memory. In this chapter, we discuss how systems organize and access file data so that it can be retrieved quickly from high-latency secondary storage devices. We also discuss how operating systems can create an interface that facilitates navigation of a user's files. Because secondary storage often contains files storing sensitive information for several users, we discuss how systems control access to file data. Many systems use files to store important information such as inventories, payrolls and account balances; we discuss how file systems can protect such data from corruption or total loss from disasters such as power and disk failures. Finally, we discuss how systems that manage large amounts of shared data can benefit from databases as an alternative to files.

13.2 Data Hierarchy

Information is stored in computers according to a **data hierarchy**. The lowest level of the data hierarchy is composed of bits. Bits are grouped together in **bit patterns** to represent all data items of interest in computer systems. There are 2^n possible bit patterns for a string of n bits.

The next level in the data hierarchy is fixed-length patterns of bits such as **bytes**, characters and words. When referring to storage, a byte is typically 8 bits. A **word** is the number of bits a processor can operate on at once. Thus, a word is 4 bytes on a 32-bit processor and 8 bytes on a 64-bit processor.

Characters map bytes (or groups of bytes) to symbols such as letters, numbers, punctuation and new lines. Many systems use 8-bit characters and thus can have 2^8 , or 256, possible characters in their **character sets**. The three most popular character sets in use today are **ASCII (American Standard Code for Information Interchange)**, **EBCDIC (Extended Binary-Coded Decimal Interchange Code)** and **Unicode®**.

ASCII stores characters as 8-bit bytes and thus can have 256 possible characters in its character set. Due to ASCII's small character size, it does not support international character sets. EBCDIC is often used for representing data in main-frame computer systems, particularly systems developed by IBM; it also stores characters as 8-bit bytes.¹

Unicode is an internationally recognized standard that is popular in Internet and multilingual applications. Its goal is to use a unique number to represent every character in all the world's languages.² Unicode provides 8-, 16- and 32-bit representations of its character set. To simplify conversion from ASCII to Unicode characters, the 8-bit representation of Unicode, called UTF-8 (Unicode character set Translation Format-8 bit), corresponds directly to the ASCII character set. HTML files are typically encoded using UTF-8. UTF-16 and UTF-32 each provide larger character sets, enabling applications to store information containing characters

from multiple alphabets, such as Greek, Cyrillic, Chinese and a great many others. However, they require larger files to store the same number of characters when compared to UTF-8. For example, the 12-character string "Hello, world" requires 12 bytes of storage using 8-bit characters, 24 bytes using 16-bit characters and 48 bytes using 32-bit characters.

A **field** is a group of characters (e.g., a person's name, street address or telephone number). A **record** is a group of fields. A student record may contain, for example, separate fields for identification number, name, address, telephone number, cumulative grade point average, major field of study, expected date of graduation and so on. A file is a group of related records. For example, a student file might contain one record for each student in a university; a payroll file might contain one record for every employee in a company. The highest level of the data hierarchy is a file system or database. File systems are collections of files, and databases are collections of data (database systems are discussed in Section 13.12, Database Systems).

The term **volume** is a unit of data storage that may hold multiple files. A physical volume is limited to one storage device; a logical volume—such as one that might be used in a virtual machine—could be draped across many devices. Examples of volumes include CDs, DVDs, tapes and hard disks.

Self Review

1. What are the trade-offs of large character sets?
2. How many possible character can be stored using a 16-bit, 32-bit and 64-bit character set? Why do you suppose that 64-bit character sets are not implemented?

Ans: 1) Large character sets such as Unicode enable users to store and transmit data in multiple languages. However, large character sets require a large number of bits to represent each character, which increases the size of data they store. 2) A 16-bit character set can represent 2^{16} , or 65,536, possible characters; a 32-bit character set can represent 2^{32} , or over 4 billion characters; a 64-bit character set can represent 2^{64} , or over 16 quintillion characters. As yet, 64-bit character sets have not been implemented, because they consume a significant amount of space per character to provide a range of characters far beyond what users are likely to need well into future.

13.3 Files

A file is a named collection of data that may be manipulated as a unit by operations such as

- **open**—Prepare a file to be referenced.
- **close**—Prevent further reference to a file until it is reopened.
- **create**—Create a new file.
- **destroy**—Remove a file.
- **copy**—Copy the contents of one file to another.
- **rename**—Change the name of a file.
- **list**—Print or display the contents of a file.

Individual data items within the file may be manipulated by operations like

- **read**—Copy data from a file to a process's memory.
- **write**—Copy data from a process's memory to a file.
- **update**—Modify an existing data item in a file.
- **insert**—Add a new data item to a file.
- **delete**—Remove a data item from a file.

Files may be characterized by attributes such as

- **size**—the amount of data stored in the file.
- **location**—the location of the file (in a storage device or in the system's logical file organization).
- **accessibility**—restrictions placed on access to file data.
- **type**—how the file data is used. For example, an executable file contains machine instructions for a process. A data file may specify the application that is used to access its data.
- **volatility**—the frequency with which additions and deletions are made to a file.
- **activity**—the percentage of a file's records accessed during a given period of time.

Files can consist of one or more records. A **physical record** (or **physical block**) is the unit of information actually read from or written to a storage device. A **logical record** (or **logical block**) is a collection of data treated as a unit by software. When each physical record contains exactly one logical record, the file is said to consist of **unblocked records**. When each physical record may contain several logical records, the file is said to consist of blocked records. In a file with fixed-length records, all records are the same length; the block size is ordinarily an integral multiple of the record size. In a file with variable-length records, records may vary in size up to the block size.

Sel Review

1. Compare physical records to logical records.
2. Why do you suppose variable-length records incur more storage overhead than fixed-length records?

Ans: 1) A physical record corresponds to the unit of information read from or written to a storage device. A logical record corresponds to a collection of data treated as a unit by software. 2) The system needs to determine the length of each record. Two common ways to do this are to end each record with an end-of-record marker, or to precede each record with a length field—each of these takes space. A system that processes fixed-length records need have the length recorded only once.

13.4 File Systems

A **file system** organizes files and manages access to data.³ File systems are responsible for:

- **File management**—providing the mechanisms for files to be stored, referenced, shared, and secured.
- **Auxiliary storage management**—allocating space for files on secondary or tertiary storage devices.
- **File integrity mechanisms**—ensuring that the information stored in a file is uncorrupted. When file integrity is assured, files contain only the information that they are intended to have.
- **Access methods**—how the stored data can be accessed.

The file system is concerned primarily with managing secondary storage space, particularly disk storage, but it can access file data stored on other media (e.g., main memory).

File systems enable users to create, modify, and delete files; they should also be able to structure files in a manner most appropriate for each application and initiate data transfer between files. Users should also be able to share each other's files in a carefully controlled manner to build upon each other's work. The mechanism for sharing files should provide various types of controlled access such as **read access**, **write access**, **execute access** or various combinations of these.

File systems should exhibit **device independence**—users should be able to refer to their files by **symbolic names** rather than having to use **physical device names**. Symbolic names are logical, user-friendly names, such as `myDirectory:myFile.txt`. Physical device names specify where a file can be found on a device, e.g., disk 2, blocks 782-791. Symbolic names allow file systems to give users a **logical view** of their data by assigning meaningful names to files and file operations. A **physical view** is concerned with the layout of file data on its storage device and the device-specific operations that manipulate the data. The user should not have to be concerned with the particular devices on which data is stored, the form the data takes on those devices or the physical means of transferring data to and from them.

Designing a file system requires knowledge of the user community, including the number of users, the average number and size of files per user, the average duration of user sessions, the nature of applications to be run on the system, and the like. These factors must be carefully considered to determine the most appropriate file organizations and directory structures.

To prevent either accidental loss or malicious destruction of information, file systems should also provide **backup** capabilities that facilitate the creation of redundant copies of data and **recovery** capabilities that enable users to restore any lost or damaged data. In sensitive environments in which information must be kept secure and private, such as in electronic funds transfer systems, criminal records systems, medical records systems, and so on, the file system may also provide **encryp-**

tion and **decryption** capabilities. This makes information useful only to its intended audience—those who possess the decryption keys (encryption and decryption are discussed in Section 19.2, Cryptography; see the Operating Systems Thinking feature, Encryption and Decryption).

Self Review

1. (T/F) File systems manage data only on secondary storage.
2. In what ways is file system management similar to virtual memory management?

Ans: 1) False. File systems manage files, which represent named collections of data that can be stored on any medium, including main memory. **2)** File system management entails allocating storage, hiding the physical view of storage from applications and controlling access to storage.

13.4.1 Directories

Consider a large-scale timesharing system supporting a large community of users. Each user may have several accounts; each account may have many files. Some files may be small, such as e-mail messages. Other files may be large, such as a master list of parts in an inventory control application.

It is common for user accounts to contain hundreds and even thousands of files. With a user community of several thousand users, a system's disks might easily contain millions of files. These files need to be accessed quickly to limit response times.

To organize and quickly locate files, file systems use **directories**, which are file-containing the names and locations of other files in the file system. Unlike other files, a directory does not store user data. Figure 13.1 lists several common directory fields.



Operating Systems Thinking

Encryption and Decryption

Years ago information processed by a computer was largely confined to one local computer system and access to it could be tightly controlled. Operating systems running on today's high-powered computer systems pass massive amounts of information between computers on networks.

especially the Internet. Transmission media are insecure and vulnerable, so to protect that information, operating systems software often provides capabilities for encryption and decryption. Both of these operations can be so processor intensive, that it was impractical to employ them

on a wide scale years ago. As processor power continues to dramatically increase, more systems will incorporate encryption and decryption into more applications, especially applications that involve vulnerable network transmission.

Directory Field	Description
Name	Character string representing the file's name.
Location	Physical block or logical location of the file in the file system (i.e., a pathname).
Size	Number of bytes consumed by the file.
Type	Description of the file's purpose (e.g., data file or directory file).
Access time	Time the file was last accessed.
Modified time	Time the file was last modified.
Creation time	Time the file was created.

Figure 13.1 | Directory file contents example. ^{4, 5, 6}

Single-Level File System

The simplest file system organization is a **single-level** (or **flat**) directory structure. In this implementation, the file system stores all of its files using one directory.⁷ In a single-level file system, no two files can have the same name. Because most environments contain a large number of files, many of which use the same name, single-level file systems are rarely implemented.

Hierarchically Structured File System

A more appropriate file system for most environments may be organized as follows (Fig. 13.2). A root indicates where on the storage device the root directory begins. Directories are files that can point to other directories and files. In Fig. 13.2, the root directory points to the various user directories. A user directory contains an entry for each of that user's files; each entry points to the location of the corresponding file on the storage device.

File names need be unique only within a given user directory. In such **hierarchically structured file systems**, each directory may contain several subdirectories but no more than one parent directory. The name of a file is usually formed as the **pathname** from the root directory to the file. For example, in a two-level file system with users SMITH, JONES and DOE, in which JONES has files PAYROLL and INVOICES, the pathname for file PAYROLL might be formed as ROOT:JONES:PAYROLL. In this example, ROOT indicates the root directory, and the use of a colon (:) delimits different pieces of the pathname.

Hierarchical file systems are implemented by most general-purpose file systems, but the name of the root directory and the type of delimiter can vary between file systems. A Windows file system's root directory is specified by a letter followed by a colon (e.g., C:), and a UNIX-based file systems uses a slash (/). Windows systems use a backslash (e.g., C:\Jones\Payroll) as a delimiter and UNIX-based systems use a slash (e.g., /jones/payroll). Various Linux and Windows XP file systems are considered in depth in the case studies (see Section 20.7.3, Second Extended File System (ext2fs), and Section 21.8.2, NTFS).

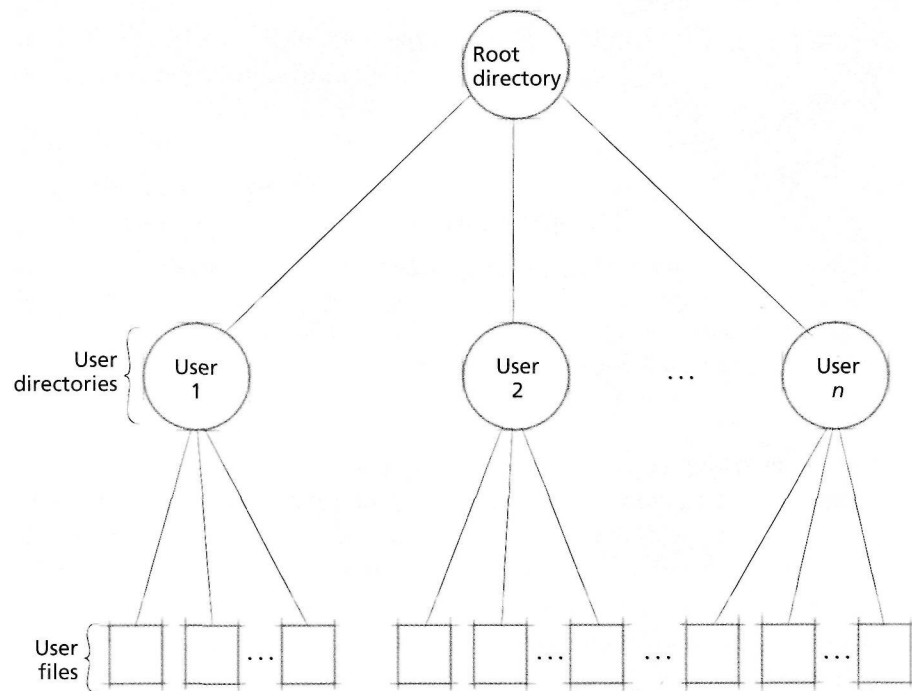


Figure 13.2 | Two-level hierarchical file system.

Relative Pathnames

Many file systems support the notion of a **working directory** to simplify navigation using pathnames. The working directory (represented by the directory entry "." in Windows and UNIX-based file systems) enables users to specify a pathname that does not begin at the root directory. For example, suppose the current working directory has been set to /home/hmd/ in the file system in Fig. 13.3. The **relative path** to /home/hmd/os/chapter13 would be os/chapter13.⁸ This feature reduces the size of the pathname when accessing files. When a file system encounters a relative pathname, it forms the **absolute path** (i.e., the path beginning at the root) by concatenating the working directory and the relative path. The file system then traverses the directory structure to locate the requested file.

The file system typically maintains a reference to the working directory's **parent directory**, the directory one level higher in the file system hierarchy. For example, in Fig. 13.3, home is the parent directory for the hmd, pjd and drc directories. In Windows and UNIX-based systems, "." is a reference to the parent directory.⁹

Links

A **link** is a directory entry that references a data file or directory that is typically located in a different directory.¹⁰ Users often employ links to simplify file system

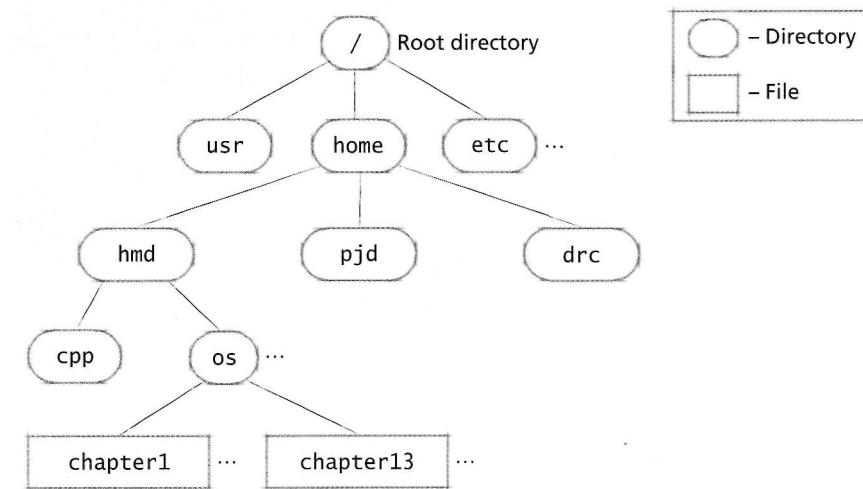


Figure 13.3 | Example hierarchical file system contents.

navigation and to share files. For example, suppose that a user's working directory is `/home/drc/` and the user shares another user's files located in the `/home/hmd/os/` directory. By creating a link to `/home/hmd/os/` in the user's working directory, the user can access the shared files simply using the relative pathname `os/`.

A **soft link** (also called a symbolic link in UNIX-based systems, a "shortcut" on Windows systems and "alias" on MacOS systems) is a directory entry containing the pathname for another file. The file system locates the target of the soft link by traversing the directory structure using the specified pathname.¹¹

A **hard link** is a directory entry that specifies the location of the file (typically a block number) on the storage device. The file system locates a hard link's file data by directly accessing the physical block it references.¹²

Figure 13.4 illustrates the difference between soft links and hard links. Files `foo` and `bar` are located at block numbers 467 and 843, respectively. The directory entry `foo_hard` is a hard link because it specifies the same block number (467) as `foo`'s directory entry. The directory entry `foo_soft` is a soft link because it specifies the pathname for `foo` (in this case, `./foo`).

Recall from Section 12.9 that disk reorganization and defragmentation can be used to improve disk performance. During such operations, the physical location of a file can change, requiring the file system to update the file's location in its directory entry. Because a hard link specifies a physical location of a file, the hard link references invalid data when the physical location of the corresponding file changes. To address this issue, a file system can store in a file a pointer to each of its hard links. When the file's physical location changes, the file system can use these pointers to find hard links that require updating.

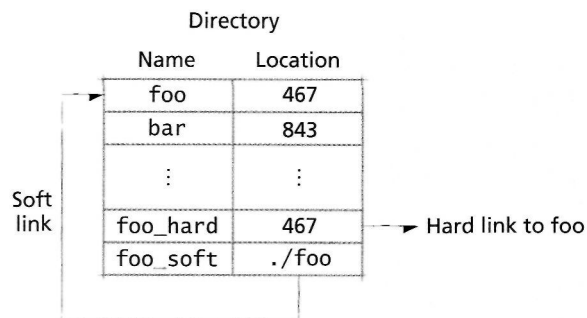


Figure 13.4 | Links in a file system.

Because soft links store the logical location of the file in the file system, which does not change during defragmentation or disk reorganization, they do not require updating when file data is moved. However, if a user moves a file to a different directory or renames the file, any soft links to that file are no longer valid. This behavior can be useful in some cases. For example, a user may wish to replace a program file with a new version of the program with the same name. In this case, the file that was originally linked can be moved, renamed or replaced, but the soft link will continue to reference a valid file (the new version of the program). Traditionally, file systems do not update soft links when a file is moved to a different directory.

When a user destroys a link to a file, the file system must determine whether to destroy the corresponding file as well. For this purpose, file systems typically maintain a count of the number of hard links to a file. When the count reaches zero, the file system contains no references to the file data, so the file can be safely removed. Because soft links do not reference the physical location of file data, they are not considered when determining whether a file should be destroyed.

SelfReview

1. Why are single-level file systems inappropriate for most systems?
2. Describe the difference between hard links and soft links.

Ans: 1) The reason is that most systems require that the file system support multiple files of the same name, a feature that single-level file systems do not provide. 2) Hard links are directory entries that specify the location of a file on its storage device; soft links specify a file's pathname. Hard links reference the same data even if the file name changes. Soft links locate a file according to its logical location in the file system. Thus, the data referenced by a soft link's pathname can change when a new file is assigned that pathname.

13.4.2 Metadata

Most file systems store data other than user data and directories, such as the locations of a storage device's free blocks (to ensure that new file data does not over-

write blocks that are being used) and the time at which a file was last modified (for accounting purposes). This information, called **metadata**, protects the integrity of the file system and cannot be modified directly by users.

Before a file system can access data, its storage device is typically **formatted**. Formatting is a system-dependent operation, but typically entails inspecting the storage device for any unusable sectors, erasing any data on the device and creating the file system's root directory. Many file systems also create a **superblock** to store information that protects the integrity of the file system. A superblock might contain:

- a **file system identifier** that uniquely identifies the type of file system
- the number of blocks in the file system
- the location of the storage device's free blocks
- the location of the root directory
- the date and time at which the file system was last modified
- information indicating whether the file system needs to be checked (e.g., due to a system failure that prevented buffered data from being written to secondary storage).^{13,14,15}

If the superblock is corrupted or destroyed, the file system might be unable to access file data. Subtle errors in superblock data (such as the location of the storage device's free blocks) could cause the file system to overwrite existing file data. To reduce the risk of lost data, most file systems distribute redundant copies of the superblock throughout the storage device. Thus, the file system can use redundant copies of the superblock to determine whether the primary superblock is damaged and, if so, replace it.

File Descriptors

When a file is opened, the operating system first locates information about the file by traversing the directory structure. To avoid further (possibly lengthy) traversals, the system maintains a table in main memory that keeps track of open files. In many systems, the file open operation returns a **file descriptor**, a non-negative integer that indexes into the open-file table. From this point on, access to the file is directed through the file descriptor.

The open-file table often contains **file control blocks**. These specify the information that the system needs to manage a file, sometimes called **file attributes**. They are highly system-dependent structures. A typical file control block might include a file's

- symbolic name
- location in secondary storage
- organization structure (e.g., sequential, direct access, and so on)
- device type (e.g., hard disk, CD-ROM)
- access control data (such as which users can access the file and the type of access that is permitted)

- type (data file, object program, C source program, and so on)
- disposition (permanent vs. temporary)
- creation date and time
- date and time last modified
- access activity counts (number of reads, for example).

Ordinarily, file control blocks are maintained on secondary storage. They are brought to main memory when a file is opened to improve the efficiency of file operations.

Self Review

1. Why should the file system maintain redundant copies of critical metadata such as superblocks?
2. Why do file systems prevent users from accessing metadata directly?

ANS: 1) Because metadata such as superblocks store information that identify the file system and provide the location of its files and free space, file system data could be lost if the superblock is damaged. Maintaining multiple copies of the superblock data enables the system to recover if the superblock is damaged, which reduces the risk of data loss. **2)** If access were not tightly controlled, accidental misuse of file system metadata could lead to inconsistencies and loss of data.

13.4.3 Mounting

Users often require access to information that is not part of the native file system (i.e., the file system that is permanently mounted on a particular system and whose root is referenced by the root directory). For example, many users store data on a second hard disk, DVD or another workstation in a network in computers. For this reason, operating systems provide the ability to **mount** multiple file systems. Mounting combines multiple file systems into one namespace—a set of files that can be identified by a single file system. The unified namespace allows users to access data from different locations as if all files were located inside the native file system.¹⁶

The mount command assigns a directory, called the **mount point**, in the native file system to the root of the mounted file system. Early Windows file systems provided a flat mounting structure; each mounted file system was assigned a letter and was located at the same level of the directory structure. Typically for example, the file system containing the operating system was mounted at C: and the next file system at D:.

UNIX-compatible file systems (such as the UNIX File System, the Fast File System and the Second Extended File System), and versions of Microsoft's NTFS 5.0 and greater, feature mount points that can be located anywhere in the file system. The contents of the native file system's directory at the mount point are temporarily hidden while another file system is mounted at that directory.¹⁷⁻¹⁸

In UNIX systems, some file systems are mounted at one of the directories located in /mnt/. Consider in Fig. 13.5. The mount command places the root of file

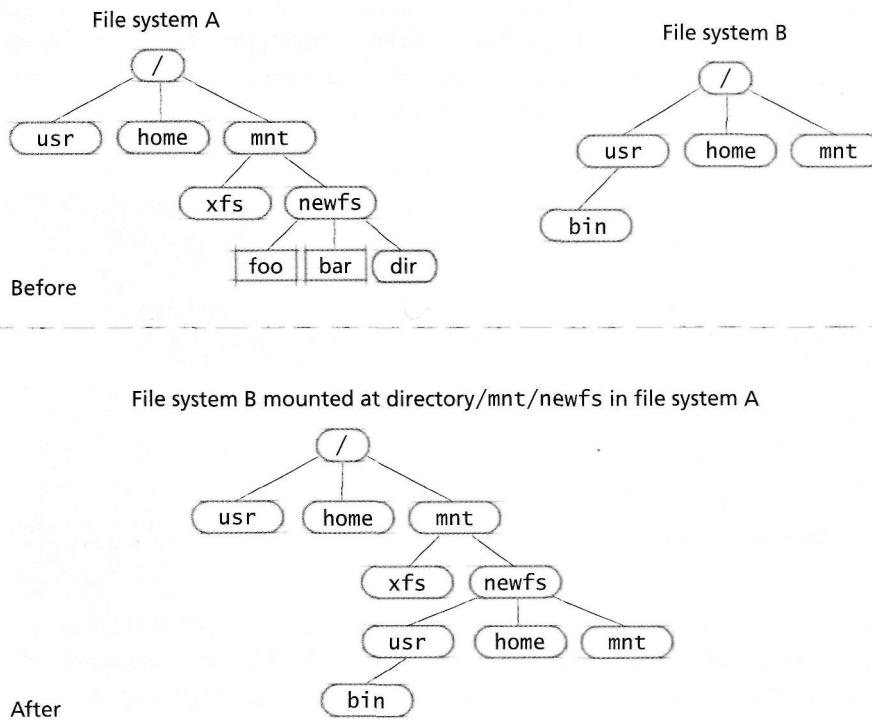


Figure 13.5 | Mounting a file system.

system B over the directory `/mnt/newfs/` of file system A. After the mount operation, files from file system B can be accessed from `/mnt/newfs/` in file system A. For example, one can access the file system B's `/usr/bin/` directory from file system A's `/mnt/newfs/usr/bin/` directory. When a file system is mounted, the contents of the directory at the mount point are unavailable to users until the file system is unmounted.

File systems manage mounted directories with **mount tables**. These contain information about mount point pathnames and the device that stores each mounted file system. When the native file system encounters a mount point, it uses the mount table to determine the device and type of file system that is mounted at that directory. Most operating systems support multiple file systems for removable storage, such as Universal Disk Format (UDF) for DVDs and the ISO 9660 file system for CDs. Once the operating system has determined the device's file system type, it uses the appropriate file system to access the file on the specified device. The unmount command allows the user to disconnect mounted file systems. This call updates the mount table and enables users to access any files that were hidden by the mounted system.¹⁹

Users can create soft links to files in mounted file systems. When the file system encounters the soft link, it uses the specified pathname to traverse the directory

structure. When the file system encounters a mount point, it begins the directory traversal from the root of the file system mounted at the mount point. A hard link, however, specifies a block number that is associated with the device that stores the link. In general, users cannot create hard links between two file systems, because they are often associated with different storage devices.

Self Review

1. (T/F) The mounted file system and the native file system must be of the same type.
2. Can a file system create a hard link to a file in a mounted file system?

Ans: 1) False. A primary advantage to mounting file systems is that they enable multiple heterogeneous file systems to be accessed via a single file system interface. 2) No, hard links reference device-specific block numbers corresponding to the file system on which they are stored, so they cannot be used to specify physical locations in other file systems.

13.5 File Organization

File organization refers to the manner in which the records of a file are arranged on secondary storage.²⁰ Several file organization schemes have been implemented:

- **Sequential**—Records are placed in physical order. The "next" record is the one that physically follows the previous record. This organization is natural for files stored on magnetic tape, an inherently sequential medium. Disk files may also be sequentially organized, but for various reasons discussed later in this chapter, records in a sequential disk file are not necessarily stored contiguously.
- **Direct**—Records are directly (randomly) accessed by their physical addresses on a direct access storage device (DASD). The application user places the records on DASD in any order appropriate for a particular application.²¹
- **Indexed sequential**—Records on disk are arranged in logical sequence according to a key contained in each record. The system maintains an index containing the physical addresses of certain principal records. Indexed sequential records may be accessed sequentially in key order, or they may be accessed directly, by a search through the system-created index.
- **Partitioned**—This is essentially a file of sequential subfiles. Each sequential subfile is called a **member**. The starting address of each member is stored in the file's directory. Partitioned files have been used to store program libraries or macro libraries.

Self Review

1. Which file organization technique is most appropriate for tape storage? Why?
2. For which media is direct file organization most appropriate?

Ans: 1) Sequential file organization is most appropriate for tape storage because it is a sequential access medium. 2) Direct file organization is most appropriate for random access devices, such as main memory and moving-head disks.

13.6 File Allocation

The problem of allocating and freeing space on secondary storage is somewhat like that experienced in main memory allocation under variable-partition multiprogramming. As files are allocated and freed, it is common for the space on secondary storage to become increasingly fragmented, with files being spread throughout widely dispersed blocks; this can cause performance problems.²² As discussed in Section 12.9, the system can perform defragmentation, but doing so while the file system is in use can lead to poor response times.

Because processes often access portions of a file sequentially, this leads us to try to store all the data in a file contiguously to improve performance. For example, users searching a file for information often use file scan options to locate the next record or the previous record. These scans should result in minimal seek activity when possible.

Spatial locality tells us that once a process has referred to a data item on a page, it is likely to reference additional data items on that page; it is also likely to reference data items on pages contiguous to that page in the user's virtual address space. Therefore it is desirable to store nonresident, logically contiguous pages of a user's virtual memory space as physically contiguous pages on secondary storage, especially if several pages are stored per physical block.

Because files often grow or shrink over time, and because users rarely know in advance how large their files will be, contiguous storage allocation systems have generally been replaced by more dynamic noncontiguous storage allocation systems. As we will see, these systems attempt to allocate parts of files contiguously to exploit locality, but enable files to change in size with minimal overhead.

Self Review

1. Compare the problem of fragmentation in file systems to that in virtual memory systems.
2. Why is it beneficial to allocate storage in blocks of a system's page size or a multiple of the page size?

Ans: 1) In both systems, fragmentation can lead to wasted storage if no data is small enough to fit in the storage "holes." Unlike main memory, which provides essentially uniform access time for each of its addresses, secondary storage devices such as disks can exhibit high access times if data is fragmented, because the storage device is likely to perform many seek operations to access file data. 2) This enables the system to exploit spatial locality by using a single I/O operation to load into memory several pages that are likely to be referenced in the near future.

13.6.1 Contiguous File Allocation

File systems that employ contiguous allocation place file data at contiguous addresses on the storage device. The user specifies the amount of space needed to

store the file in advance. If the desired amount of contiguous space is not available, the file cannot be created.

An advantage of contiguous allocation is that successive logical records typically are physically adjacent to one another. This speeds access compared to systems in which successive logical records are dispersed throughout secondary storage, requiring additional seek operations. Locating file data is straightforward, because the directories need store only the address of the start of the file and the file's length.

A disadvantage to contiguous allocation schemes is that they exhibit the same types of external fragmentation problems inherent in memory allocation for variable-partition multiprogramming systems (see Section 9.9). Also, contiguous allocation can result in poor performance as files grow and shrink over time. If a file grows beyond the size originally specified and there are no adjacent free blocks, the file must be transferred to a new area of adequate size, leading to additional I/O operations. To provide for anticipated expansion, users might overestimate their storage needs, leading to inefficient storage allocation. Contiguous allocation is particularly useful for write-once CDs and DVDs, which do not allow files to grow or shrink over time.

Self Review

1. Explain the benefits of using a contiguous file allocation scheme.
2. Explain the disadvantages of using a contiguous file allocation scheme.

Ans: 1) Locating file data is straightforward. Also, files can be accessed quickly, because the storage device does not need to perform lengthy seeks after it locates the first block.
 2) Contiguous allocation schemes can lead to significant external fragmentation and poor performance when a file grows too large to be stored contiguously at its current location and must be moved.

13.6.2 Linked-List Noncontiguous File Allocation

Most file systems implemented on rewritable secondary storage media use noncontiguous allocation. One approach to noncontiguous file allocation is to implement a sector-based linked list. In this scheme, each directory entry points to the first sector of a file on a moving-head storage device such as a hard disk. The data portion of a sector stores the contents of the file; the pointer portion stores a pointer to the file's next sector. Because files often occupy multiple sectors, a disk's read/write head must sequentially access each file sector until it finds the requested record.

Noncontiguous allocation solves some of the problems inherent in contiguous allocation schemes, but it has its own drawbacks. Because a file's records may be dispersed throughout the disk, direct and sequential access to logical records can involve many additional seeks besides the first seek to the file. Pointers in the list structure also reduce the amount of space available for file data in each sector.

One scheme used to manage secondary storage more efficiently and reduce file-traversal overhead is called **block allocation**. In this scheme, instead of allocating individual sectors, blocks (often called **extents**) of contiguous sectors are allocated. The system tries to allocate new blocks to a file by choosing free blocks as close as possible

to existing file data blocks, preferably on the same cylinder. Each access to the file involves determining the appropriate block and sector within the block.

In block **chaining**, each entry in a directory points to the first block of a file (Fig. 13.6). The blocks comprising a file each contain two portions: a data block and a pointer to the next block. The smallest unit of allocation is a fixed-size block that ordinarily consists of many sectors. Locating a particular record requires searching the block chain until the appropriate block is found, then searching that block until the appropriate record is found. The chain must be searched from the beginning, and if the blocks are dispersed throughout the disk (which is normal), the search process can be slow, as block-to-block seeks occur. Insertion and deletion are performed simply by modifying the pointer in the previous block. Some systems use doubly linked lists to facilitate searching; the blocks are threaded both forward and backward so that a search may proceed in either direction.

Block size can significantly impact file system performance. If blocks are split between files (e.g., a file that requires 2.5 blocks occupies three blocks on disk), large block sizes can result in a considerable amount of internal fragmentation. Large block sizes, however, reduce the number of I/O operations required to access file data. Small block sizes may cause file data to be spread across multiple blocks, which tend to be dispersed throughout the disk. This could lead to poor performance. In practice, block sizes typically range from one to eight kilobytes.^{23,24,25}

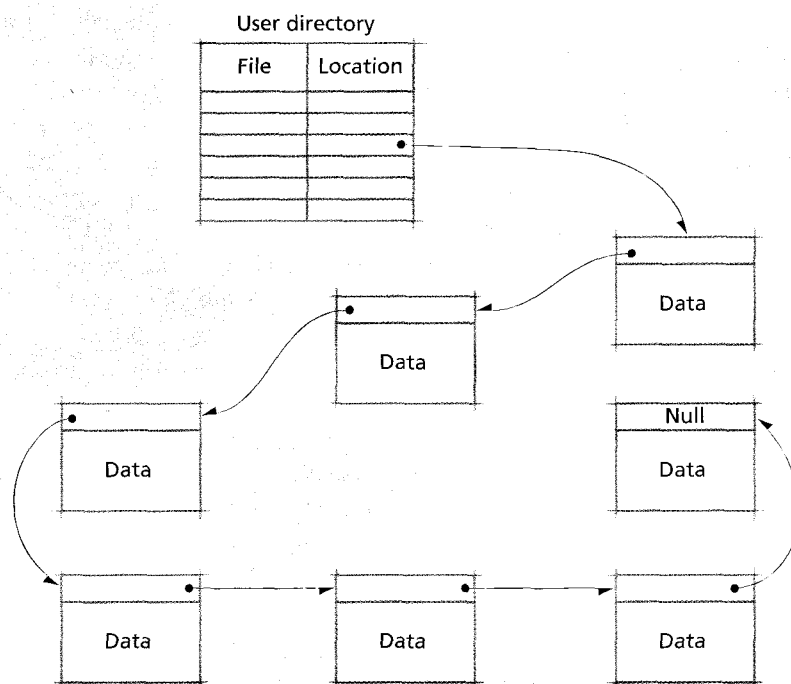


Figure 13.6 | Noncontiguous file allocation using a linked list.

Self Review

1. What is the primary disadvantage of linked-list noncontiguous file allocation?
2. What are the trade-offs between choosing a large block size and a small block size?

Ans: **1)** In the worst case, the file system must access each of the file's blocks (when using a singly linked list) or half of the file's blocks (when using a doubly linked list) to locate file data, which could lead to long access times. As we discuss in the next sections, several popular file allocation techniques place a lower limit on access times, which tends to improve performance. **2)** A large block size generally reduces the number of I/O operations required to retrieve a particular record, at the cost of wasted storage due to internal fragmentation. A small block size reduces the amount of internal fragmentation but if file data is scattered throughout a device such as a hard disk, it can lead to poor access times.

13.6.3 Tabular Noncontiguous File Allocation

Tabular noncontiguous file allocation stores pointers to file blocks contiguously in tables to reduce the number of lengthy seeks required to access a particular record (Fig. 13.7). Directory entries indicate the first block of a file. For example, the first block of file C in Fig. 13.7 is 2. The current block number is used as an index into the block allocation table to determine the location of the next block. Therefore, the value of file C's next block number is stored in location 2 in the block allocation table. In this case, file C's next block is 5. The block allocation table's entry for a file's last block stores the value null.

Because the pointers that locate file data are stored in a central location, the table can be cached so that the chain of blocks that compose a file can be traversed quickly, which improves access times. Locating the last record of a file may require the file system to follow many pointers in the block allocation table. To reduce access times, the block allocation table should be stored contiguously on disk and cached in main memory. When a file system contains a small number of blocks, this can be accomplished relatively easily. For example, a 1.44MB floppy disk using 1KB blocks contains 1,440 blocks, which can be addressed using an 11-bit number. The size of the corresponding block allocation table is the size of each block address multiplied by the number of blocks—in this case, approximately 2,000 bytes.

For file systems that contain a greater number of blocks, the size of each file allocation table entry, and thus the size of the table, is greater. For example, a 200GB disk using 4KB blocks contains 50,000,000 blocks, which can be addressed using a 26-bit number. In this case, the block allocation table consumes over 160MB. In the preceding example, the file allocation table was stored in only two blocks; however, in this case the file allocation table is spread across tens of thousands of blocks, which could lead to fragmentation. If file data is dispersed across the storage device, the file's table entries will be spread throughout the block table. Thus, the system might need to load into memory several blocks of the block allocation table, which could lead to poor access times. Also, if the block allocation table is cached, it can consume significant memory.

A popular implementation of tabular noncontiguous file allocation is Microsoft's **FAT file system**. Microsoft first incorporated a **file allocation table**

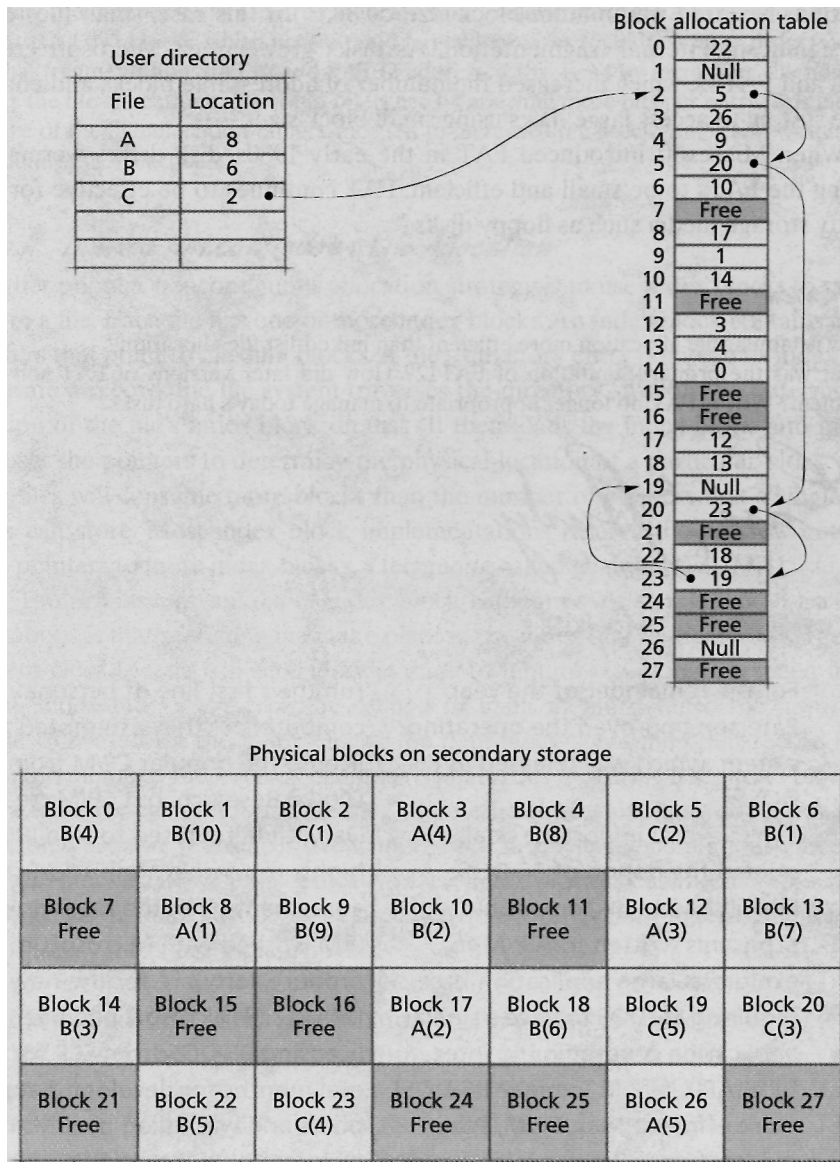


Figure 13.7 | *Tabular noncontiguous file allocation.*

(FAT) into its release of MS-DOS 1.0 (see the Mini Case Study, MS-DOS).²⁶ The FAT stores information about each block, including whether the block is currently allocated and the number of the next block in the file. The first version of the FAT file system, called FAT12, allocated 12 bits per entry in the FAT. This meant that a disk managed by FAT12 could contain no more than 2^{12} (4,096) blocks. This was sufficient for small disks containing few files, but FAT12 resulted in significant

wasted memory for larger disks. For example, to address an entire 64MB disk, the file system required a minimum block size of 8KB. In this case, small files could incur significant internal fragmentation. As disks grew larger, Microsoft created FAT16 and FAT32, which increased the number of addressable blocks and enabled the file system to access large disks using small block sizes.²⁷⁻²⁸

When Microsoft introduced FAT in the early 1980s, disk drives were small, allowing the FATs to be small and efficient. FAT continues to be effective for low-capacity storage media such as floppy disks.²⁹

SelfReview

1. How is tabular file allocation more efficient than linked-list file allocation?
2. What was the primary limitation of FAT12? How did later versions of FAT solve this problem? Why is FAT no longer appropriate to manage today's hard disks?



Mini Case Study

MS-DOS

Microsoft **MS-DOS** was based on a disk operating system written by Tim Paterson for Seattle Computing Products (SCP) in 1980.³⁰ At the time, SCP needed an operating system for their memory board for the new Intel 16-bit 8086 processor.³¹ Digital Research's (CP/M) Control Program for Microcomputers was the major microcomputer operating system at the time, but an updated version for the 8086 processor was not ready when SCP's memory boards went on sale.³² Paterson was asked to write a new operating system, which was named QDOS (Quick and Dirty Operating System). As the name implies, it was completed in just a few months but had not been thoroughly tested and debugged.

For the remainder of the year, Paterson improved the operating system, which was released as 86-DOS.³³

Several important goals guided the design of 86-DOS. First, it had to be compatible with programs written for CP/M to exploit its large application base, requiring 86-DOS to have a similar application programming interface (API).^{34, 35, 36} To make 86-DOS more efficient than CP/M, Paterson wrote it in assembly language and incorporated the File Allocation Table (FAT) disk management system, based on Microsoft's Stand-Alone Disk BASIC system (see Section 13.6.3, Tabular Non-contiguous File Allocation).^{37, 38, 39}

In 1981, IBM solicited vendors for an operating system to

run their first line of personal computers.⁴⁰ They attempted to license the popular CP/M from Digital Research, but CP/M creator Gary Kildall refused to sign IBM's highly restrictive nondisclosure agreements.⁴¹ In the meantime, IBM worked with Microsoft to produce software for the new IBM PCs.⁴² Microsoft had been licensing 86-DOS from SCP for several months for development purposes and was asked by IBM to purchase all rights to DOS.^{43, 44, 45} SCP accepted the offer and 86-DOS, renamed MS-DOS, became the de facto operating system for the IBM Personal Computer and compatible microcomputers.⁴⁶

Ans: 1) Block allocation tables can store the locations of the file system's data contiguously, reducing the number seeks required to locate a particular record. 2) FAT12 could not address more than 4,096 blocks, which necessitated large block sizes to address large disks, leading to internal fragmentation. FAT16 and FAT32 addressed this issue by increasing the number of blocks the file allocation table could reference by allowing more bits per entry. This increased the size of the file allocation table, which led to increased file access times and wasted memory in file system caches.

13.6.4 Indexed Noncontiguous File Allocation

Another popular noncontiguous allocation strategy is to use **index blocks** to point to data in a file. Each file has one or more index blocks. An index block contains a list of pointers that point to file data blocks. A file's directory entry points to its index block. To locate a record, the file system traverses the directory structure to determine the location of the file's index block on disk. It then loads the index block into memory and uses the pointers to determine the physical location of a particular block. Often, large files will consume more blocks than the number of pointers that a single index block can store. Most index block implementations reserve the last few entries to store pointers to more index blocks, a technique called **chaining** (Fig. 13.8).

The primary advantage of index block chaining over simple linked-list implementations is that searching may take place in the index blocks themselves. They may be kept close together in secondary storage to minimize seeking. To speed file traversal, index blocks are typically cached in main memory. Once the appropriate record is located via the index blocks, the data block containing that record is read into main memory. Index block chaining is analogous to storing a separate block allocation table for each file, which can be more efficient than systemwide block allocation tables, because references to each file's blocks are stored contiguously in each of its index blocks. File systems typically place index blocks near the data blocks they reference, so the data blocks can be accessed quickly after their index block is loaded.

Index blocks are called **inodes** (i.e., index nodes) in UNIX-based operating systems.⁴⁷ A file's inode stores the file's attributes, such as its owner, size, time of creation and the time of last modification. It also stores the addresses of some of the file's data blocks and pointers to continuation index blocks called **indirect blocks**. Inode structures support up to three levels of indirect blocks (Fig. 13.9). The first indirect block points to data blocks; these data blocks are singly indirect. The second indirect block contains pointers that reference only other indirect blocks. These indirect blocks point to data blocks that are doubly indirect. The third indirect block points only to other indirect blocks that point only to more indirect blocks that point to data blocks; these data blocks are triply indirect. The power of this hierarchical structure is that it places a relatively low limit on the maximum number of pointers that must be followed to locate file data—it enables inodes to locate any data block by following at most four pointers (the inode and up to three levels of indirect blocks).

Inodes are investigated in detail in the Linux case study in Section 20.7.3, Second Extended File System (ext2fs). Another file system that uses index blocks is

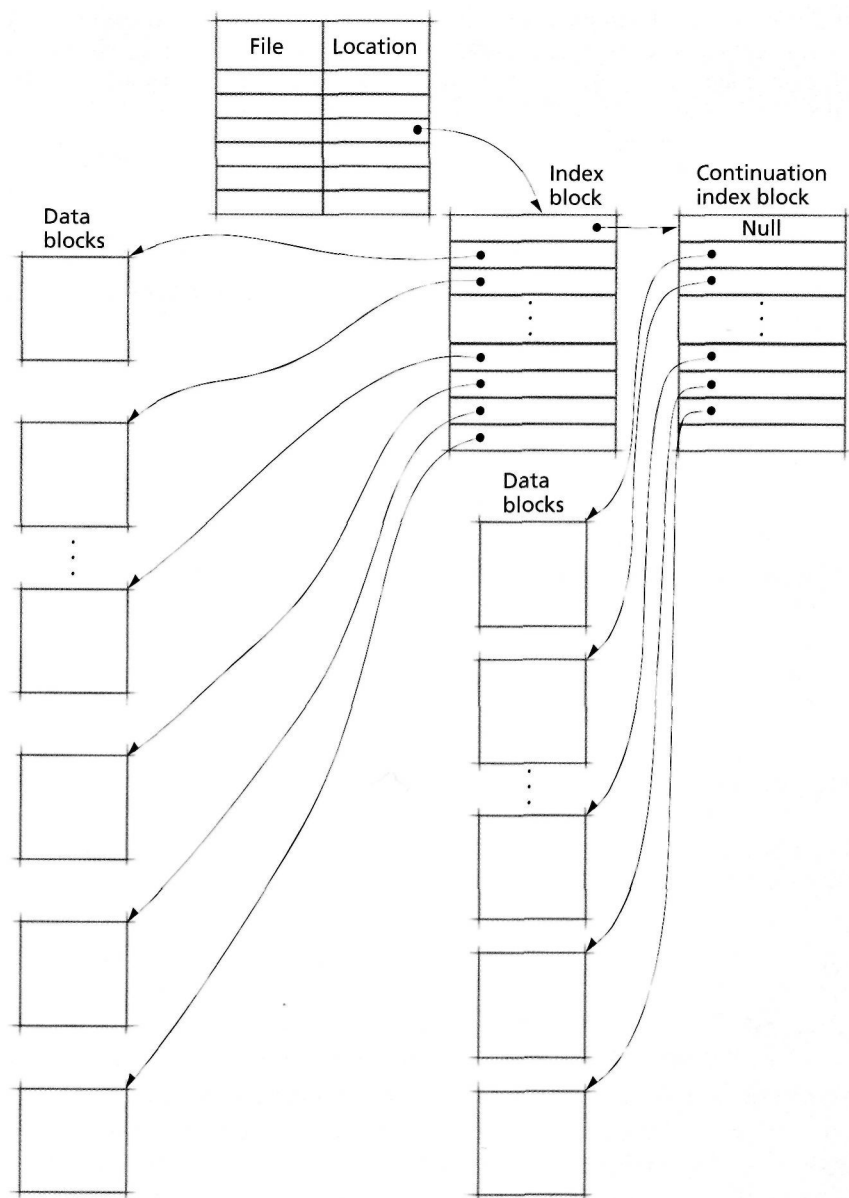


Figure 13.8 | Index block chaining.

Microsoft's NTFS, which is discussed in the Windows XP case study in Section 21.8.2, NTFS. Indexed noncontiguous allocation yields good performance with low overhead for many environments and has been implemented in many general-purpose computer systems.